

DATA STRUCTURE USING C

DATA STRUCTURE USING C

NAME : YUVASRI . M

DEPARTMENT : B.E CSE

DATA STRUCTURE USING C

MODULE 1 - List ADT and Linear Representations

1. Write a program to find the mean of the elements in the array.

PROGRAM:

```
#include<stdio.h>

int main()
{
    int n;
    int arr[20],sum=0;
    double mean;
    printf("Enter the number of elements in the array\n");
    scanf("%d",&n);
    printf("Enter the elements in the array\n");
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        sum+=arr[i];
    }
    mean=sum/n;
    printf("The mean of the array is %.2f\n",mean);

    return 0;
}
```

SAMPLE INPUT AND OUTPUT :

Enter the number of elements in the array

5

Enter the elements in the array

2

4

1

3

5

The mean of the array is 3.00

2. Write a program to find the median of the elements in the array

PROGRAM:

```
#include<stdio.h>
```

```
int main(){
```

```
    int n,a[20],i,j,temp;
```

```
    double median;
```

```
    printf("Enter the number of elements in the array\n");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the elements in the array\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    for(i=0;i<n-1;i++)
```

```
        for(j=i+1;j<n;j++){
```

```
            if(a[i]>a[j]){
```

```
                temp=a[i];
```

```
                a[i]=a[j];
```

```
                a[j]=temp;
```

```
            }
```

```
        }
```

```
    if(n%2==0)
```

```
        median=(a[n/2]+a[n/2-1])/2.0;
```

```
    else
```

```
median=a[n/2];  
printf("The median of the array is %.2f",median);  
return 0;  
}
```

SAMPLE INPUT AND OUTPUT:

Enter the number of elements in the array

5

Enter the elements in the array

2

4

1

3

5

The median of the array is 3.00

3. Write a program to find the mode of the elements in the array.

PROGRAM:

```
#include<stdio.h>  
  
int main(){  
    int n,i,j,mode,maxCount=0;  
    printf("Enter the number of elements in the array\n");  
    scanf("%d",&n);  
    int a[n];  
    printf("Enter the elements in the array\n");  
    for(i=0;i<n;i++){  
        scanf("%d",&a[i]);  
    }  
    for(i=0;i<n;i++){  
        int count=0;
```

```

        for(j=0;j<n;j++){
            if(a[i]==a[j]){
                count++;
            }
        }
        if(count>maxCount){
            maxCount=count;
            mode=a[i];
        }

    }

    if(maxCount==1){
        printf("The mode of the array is none\n");
    }
    else{
        printf("The mode of the array is \n%d\n",mode);
    }
    return 0;
}

```

SAMPLE INPUT AND OUTPUT 1:

Enter the number of elements in the array

5

Enter the elements in the array

2

4

1

3

5

The mode of the array is none

SAMPLE INPUT AND OUTPUT 2 :

Enter the number of elements in the array

5

Enter the elements in the array

2

4

2

3

5

The mode of the array is

2

4. Write a program to find the range of the elements in the array.

PROGRAM:

```
#include<stdio.h>
```

```
int main(){
```

```
    int n,a[20],i;
```

```
    int max,min,range;
```

```
    printf("Enter the number of elements in the array\n");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the elements in the array\n");
```

```
    for(i=0;i<n;i++){
```

```
        scanf("%d",&a[i]);
```

```
        max=a[0];
```

```
        min=a[0];
```

```
    }
```

```
    for(i=1;i<n;i++){
```

```

        if(a[i]>max)
            max=a[i];
        if(a[i]<min)
            min=a[i];

    }

    range=max-min;
    printf("The range of the array is %d",range);
    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the number of elements in the array

5

Enter the elements in the array

2

4

1

3

5

The range of the array is 4

5. Write a program to sort the elements of an array in ascending order for the given starting position and ending position. Remaining Elements need to be sorted in descending order.

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main()
```

```
{
```

```
int*a;

int n,i,j,s,e,t;

scanf("%d",&n);

a=(int*)malloc(n*sizeof(int));

for(i=0;i<n;i++)

scanf("%d",a+i);

scanf("%d",&s);

scanf("%d",&e);


for(i=s;i<=e;i++)

for(j=i+1;j<=e;j++)

if(*(a+i)>*(a+j)){

    t=*(a+i);

    *(a+i)=*(a+j);

    *(a+j)=t;

}

for(i=0;i<n;i++)

if(i<s||i>e)

for(j=i+1;j<n;j++)

if((j<s||j>e)&&*(a+i)<*(a+j)){

    t=*(a+i);

    *(a+i)=*(a+j);

    *(a+j)=t;

}

for(i=0;i<n;i++)

printf("%d\n",*(a+i));


return 0;
```



```
}
```

SAMPLE INPUT 1:

```
7
1
7
2
5
8
3
6
2
4
```

SAMPLE OUTPUT 1:

```
7
6
2
5
8
3
1
```

6. Write a program to find the inversion count of an array.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

int main()
{
    int*a;
    int n,i,j;
    int count=0;
    printf("Enter the number of elements in the array\n");
    scanf("%d",&n);
    a=(int*)malloc(n*sizeof(int));
```

```

printf("Enter the elements in the array\n");
for(i=0;i<n;i++)
scanf("%d",a+i);
for(i=0;i<n;i++)
for(j=i+1;j<n;j++)
if(*(a+i)>*(a+j))
count++;
printf("The inversion count of the array is %d",count);

return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the number of elements in the array

5

Enter the elements in the array

2

4

1

3

5

The inversion count of the array is 3

7. Given the position of a bishop and a queen in a $n \times n$ chessboard, mark the remaining positions in the chess board as follows:

'*' --- if it is under attack from bishop

'\$' --- if it is under attack from queen

'%' --- if it is under attack from both queen and bishop.

',' --- if it is not under attack.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

int main()
{
    int n;
    int bx,by,qx,qy;
    int i,j;
    scanf("%d",&n);
    scanf("%d",&bx);
    scanf("%d",&by);
    scanf("%d",&qx);
    scanf("%d",&qy);
    bx--;by--;
    qx--;qy--;
    for(i=0;i<n;i++){
    for(j=0;j<n;j++){
        int bishop_attack=abs(i-bx)==abs(j-by);
        int queen_attack=(i==qx||j==qy||abs(i-qx)==abs(j-qy));
        if(i==bx && j==by)
            printf("B");
        else if(i==qx && j==qy)
            printf("Q");
        else if(bishop_attack && queen_attack)
            printf("%%");
        else if(bishop_attack)
            printf("*");
        else if(queen_attack)
            printf("$");
```

```

        else
            printf(".");
    }
    printf("\n");
}
return 0;
}

```

SAMPLE INPUT :

```

4
2
2
3
4

```

SAMPLE OUTPUT:

```

*      $ * $
.      B $ $
%      $ % Q
.      . $ %

```

8. Write a program to just check if the given matrix is a valid sudoku solution.

PROGRAM:

```

#include<stdio.h>
#include<math.h>
int main()
{
    int N;
    scanf("%d",&N);

```

```
int a[N][N];

for(int i=0;i<N;i++){
    for(int j=0;j<N;j++){
        scanf("%d",&a[i][j]);
    }
}

int used[65];

int root=sqrt(N);

for(int i=0;i<N;i++){
    for(int k=1;k<=N;k++)used[k]=0;

    for(int j=0;j<N;j++){
        int val=a[i][j];

        if(val<1||val>N||used[val]){
            printf("no");

            return 0;
        }

        used[val]=1;
    }
}

for(int j=0;j<N;j++){
    for(int k=1;k<=N;k++)used[k]=0;

    for(int i=0;i<N;i++){
        int val=a[i][j];

        if(used[val]){
            printf("no");

            return 0;
        }

        used[val]=1;
    }
}
```

```
for(int r=0;r<N;r+=root){
    for(int c=0;c<N;c+=root){
        for(int k=1;k<=N;k++)used[k]=0;
        for(int i=r;i<r+root;i++){
            for(int j=c;j<c+root;j++){
                int val=a[i][j];
                if(used[val]){
                    printf("no");
                    return 0;
                }
                used[val]=1;
            }
        }
    }
}
printf("yes");
return 0;
}
```

SAMPLE INPUT 1:

```
4
1 2 3 4
3 4 1 2
2 1 4 3
4 3 2 1
```

SAMPLE OUTPUT 1:

yes

SAMPLE INPUT 2:

4

1 2 3 4

2 1 4 3

3 4 1 2

4 3 2 1

SAMPLE OUTPUT 2:

No

9. Given a sorted sequence of n integers $S = s_1, s_2, \dots, s_n$ and a sorted sequence of m integers $Q = q_1, q_2, \dots, q_m$, Write a program to print in the ascending order all such s_i that does not belong to Q.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

int *createArray(int size);

void readArray(int *arr, int size);

void printArray(int *arr, int size);

void formNewArray(int *arr1, int size1, int *arr2, int size2, int *arr3, int *size3);

int main()
{
    int n, m, size3 = 0;
    int *arr1, *arr2, *arr3;
    printf("Enter the number of elements in the first array\n");
    scanf("%d", &n);
    arr1 = createArray(n);
    printf("Enter the elements in the first array\n");
    readArray(arr1, n);
    printf("Enter the number of elements in the second array\n");
```

```

scanf("%d", &m);
arr2 = createArray(m);
printf("Enter the elements in the second array\n");
readArray(arr2, m);
arr3 = createArray(n);
formNewArray(arr1, n, arr2, m, arr3, &size3);
printf("The first array is\n");
printArray(arr1, n);
printf("The second array is\n");
printArray(arr2, m);
if (size3 > 0)
{
    printf("The new array formed is\n");
    printArray(arr3, size3);
}
else
{
    printf("No item in the new array\n");
}
return 0;
}

int *createArray(int size)
{
    int *arr;
    arr = (int *)malloc(size * sizeof(int));
    return arr;
}

void readArray(int *arr, int size)
{

```



```
int i;
for (i = 0; i < size; i++)
{
    scanf("%d", arr + i);
}
}

void printArray(int *arr, int size)
{
    int i;
    for (i = 0; i < size; i++)
    {
        printf("%d ", *(arr + i));
    }
    printf("\n");
}

/* Fill Your Code Here */

void formNewArray(int *arr1, int size1, int *arr2, int size2, int *arr3, int *size3)
{
    int i, j, found;
    *size3 = 0;
    for (i = 0; i < size1; i++)
    {
        found = 0;
        for (j = 0; j < size2; j++)
        {
            if (arr1[i] == arr2[j])
            {
                found = 1;
                break;
            }
        }
    }
}
```

```
        }}  
if (found == 0)  
{  
    arr3[*size3] = arr1[i];  
    (*size3)++;  
}  
}  
}
```

SAMPLE INPUT AND OUTPUT 1:

Enter the number of elements in the first array

8

Enter the elements in the first array

1 2 3 4 5 6 7 8

Enter the number of elements in the second array

5

Enter the elements in the second array

1 2 3 4 5

The first array is

1 2 3 4 5 6 7 8

The second array is

1 2 3 4 5

The new array formed is

6 7 8

SAMPLE INPUT AND OUTPUT 2:

Enter the number of elements in the first array

5

Enter the elements in the first array

1 2 3 4 5

Enter the number of elements in the second array

5

Enter the elements in the second array

1 2 3 4 5

The first array is

1 2 3 4 5

The second array is

1 2 3 4 5

No item in the new array

MODULE 2 – Linked List And Applications

1. Linked List : Append

Write a program to *append* a node to the Linked List.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

struct node

{

    int data;

    struct node*link;

};

void append(struct node **head, int val)

{

    struct node *newNode,*temp;

    newNode=(struct node*)malloc(sizeof(struct node));

    newNode->data=val;

    newNode->link=NULL;

    if(*head == NULL)

        *head=newNode;

    else

    {

        temp=*head;

        while(temp->link!=NULL)

            temp=temp->link;

        temp->link=newNode;

    }

}
```

```

}

void display(struct node *head)
{
    while(head!=NULL)
    {
        printf("%d ",head->data);
        head=head->link;
    }
}

int main()
{
    struct node *head=NULL;
    int val;
    char ch[5];
    do
    {
        printf("Enter the value\n");
        scanf("%d",&val);
        append(&head,val);
        printf("Do you want to add another node? Type Yes/No\n");
        scanf("%s",ch);
    }
    while(strcmp(ch, "Yes")==0);
    printf("The elements in the linked list are ");
    display(head);
    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to add another node? Type Yes/No

Yes

Enter the value

17

Do you want to add another node? Type Yes/No

Yes

Enter the value

11

Do you want to add another node? Type Yes/No

Yes

Enter the value

28

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 10 17 11 28

2.Linked List : Count

Write a program to *count* the number of nodes in the Linked List.

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node * link;
```

```
};
```

```
void append ( struct node **, int );
```

```
void display ( struct node * );
```

```
int count ( struct node * );
```

```
int main() {
```

```
    struct node *p;
```

```
    p=NULL;
```

```

int n;
char ch[10];
do {
    printf("Enter the value\n");
    scanf("%d",&n);
    append(&p,n);
    printf("Do you want to add another node? Type Yes/No\n");
    scanf("%s",ch);
}while(!strcmp(ch,"Yes"));
printf("The elements in the linked list are");
display(p);
printf("\n");
printf("The number of elements in the linked list is %d\n",count(p));
return 0;
}

void append ( struct node **q, int num ) {
    struct node*newNode,*temp;
    newNode=(struct node*)malloc(sizeof(struct node));
    newNode->data=num;
    newNode->link=NULL;
    if(*q==NULL){
        *q=newNode;
    }
    else{
        temp=*q;
        while(temp->link!=NULL)
            temp=temp->link;
        temp->link=newNode;
    }
}

```

```

void display ( struct node *q ) {
    while(q!=NULL){
        printf("%d",q->data);
        q=q->link;
    }
}

int count ( struct node * q ) {
    int c=0;
    while(q!=NULL){
        c++;
        q=q->link;
    }
    return c;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to add another node? Type Yes/No

Yes

Enter the value

17

Do you want to add another node? Type Yes/No

Yes

Enter the value

11

Do you want to add another node? Type Yes/No

Yes

Enter the value

28

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 10 17 11 28

The number of elements in the linked list is 4

3.Linked List : Add at Beginning

Write a program to add a node at the beginning of the Linked List.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

struct node {

    int data;

    struct node * link;

};

void addatbeg ( struct node **, int );

void display ( struct node * );

int main() {

    struct node *p ;

    p=NULL;

    int n;

    char ch[10];

    do {

        printf("Enter the value\n");

        scanf("%d",&n);

        addatbeg(&p,n);

        printf("Do you want to add another node? Type Yes/No\n");

        scanf("%s",ch);

    }

    while(!strcmp(ch,"Yes"));

    printf("The elements in the linked list are");

    display(p);

    printf("\n");

    return 0;}
```

```

void addatbeg ( struct node **q, int num )
{
    {
        struct node*newNode;
        newNode=(struct node*)malloc(sizeof(struct node));
        newNode->data=num;
        newNode->link=*q;
        *q=newNode;
    }
}

void display ( struct node *q )
{
    while(q!=NULL)
    {
        printf("%d",q->data);
        q=q->link;
    }
}

```

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to add another node? Type Yes/No

Yes

Enter the value

17

Do you want to add another node? Type Yes/No

Yes

Enter the value

11

Do you want to add another node? Type Yes/No

Yes

Enter the value

28

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 28 11 17 10

4.Linked List : Add after particular position

Write a program to add a node after a particular position in the Linked List.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

struct node {

    int data;

    struct node *link;

};

void append(struct node **, int);

void display(struct node *);

void addafter(struct node *, int, int);

int main() {

    struct node *p;

    p = NULL;

    int n, l;

    char ch[10];

    do {

        printf("Enter the value\n");

        scanf("%d", &n);

        append(&p, n);

        printf("Do you want to append another node? Type Yes/No\n");

        scanf("%s", ch);
```

```

    } while (!strcmp(ch, "Yes"));

    printf("The elements in the linked list are");

    display(p);

    printf("\n");

    do {

        printf("Enter the position after which you want to add another node\n");

        scanf("%d", &l);

        printf("Enter the value\n");

        scanf("%d", &n);

        addafter(p, l, n);

        printf("The elements in the linked list are");

        display(p);

        printf("\n");

        printf("Do you want to add another node after a certain position? Type Yes/No\n");

        scanf("%s", ch);

    } while (!strcmp(ch, "Yes"));

    return 0;

}

void append(struct node **q, int num) {

    struct node *nn = (struct node *)malloc(sizeof(struct node));

    struct node *temp = *q;

    nn->data = num;

    nn->link = NULL;

    if (*q == NULL) {

        *q = nn;

    } else {

        while (temp->link != NULL) {

            temp = temp->link;

        }

    }

```

```

        temp->link = nn;
    }
}

void display(struct node *q) {
    while (q != NULL) {
        printf(" %d", q->data);
        q = q->link;
    }
}

void addafter(struct node *q, int loc, int num) {
    int i;
    struct node *temp = q;
    for (i = 0; i < loc; i++) {
        if (temp == NULL) {
            printf("There are less than %d elements in the linked list\n", loc);
            return;
        }
        temp = temp->link;
    }
    if (temp == NULL) {
        printf("There are less than %d elements in the linked list\n", loc);
        return;
    }
    struct node *nn = (struct node *)malloc(sizeof(struct node));
    nn->data = num;
    nn->link = temp->link;
    temp->link = nn;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to append another node? Type Yes/No

Yes

Enter the value

17

Do you want to append another node? Type Yes/No

Yes

Enter the value

11

Do you want to append another node? Type Yes/No

Yes

Enter the value

28

Do you want to append another node? Type Yes/No

No

The elements in the linked list are 10 17 11 28

Enter the position after which you want to add another node

2

Enter the value

8

The elements in the linked list are 10 17 11 8 28

Do you want to add another node after a certain position? Type Yes/No

Yes

Enter the position after which you want to add another node

8

Enter the value

15

There are less than 8 elements in the linked list

The elements in the linked list are 10 17 11 8 28

Do you want to add another node after a certain position? Type Yes/No

No

5.Linked List : Search

Write a program to find whether an element is present in the linked list.

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>

struct node {
    int data;
    struct node * link;
};

void append ( struct node **, int ) ;
void display ( struct node * ) ;
int search (struct node *, int);
int main() {
    struct node *p ;
    p=NULL;
    int n,ele;
    char ch[10];
    do {
        printf("Enter the value\n");
        scanf("%d",&n);
        append(&p,n);
        printf("Do you want to add another node? Type Yes/No\n");
        scanf("%s",ch);
    } while(!strcmp(ch,"Yes"));
    printf("The elements in the linked list are");
    display(p);
    printf("\n");
    printf("Enter the element to be searched\n");
    scanf("%d",&ele);
    if(search(p,ele))
        printf("%d is present in the linked list\n",ele);
    else
        printf("%d is not present in the linked list\n",ele);
}
```

```

    return 0;
}

void append ( struct node **q, int num ) {
    struct node*nn,*temp;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=num;
    nn->link=NULL;
    if(*q==NULL){
        *q=nn;
    }else{
        temp=*q;
        while(temp->link!=NULL)
            temp=temp->link;
        temp->link=nn;
    }
}

void display ( struct node *q ) {
    while(q!=NULL){
        printf("%d",q->data);
        q=q->link;
    }
}

int search ( struct node *q, int e ) {
    while(q!=NULL){
        if(q->data==e)
            return 1;
        q=q->link;
    }
    return 0;}

```


SAMPLE INPUT AND OUTPUT 1:

Enter the value

5

Do you want to add another node? Type Yes/No

Yes

Enter the value

9

Do you want to add another node? Type Yes/No

Yes

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 5 9 2 8

Enter the element to be searched

2

2 is present in the linked list

SAMPLE INPUT AND OUTPUT 2:

Enter the value

5

Do you want to add another node? Type Yes/No

Yes

Enter the value

9

Do you want to add another node? Type Yes/No

Yes

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 5 9 2 8

Enter the element to be searched

12

12 is not present in the linked list

6.Linked List : Delete any element

Write a program to delete a particular element in the Linked List.

SAMPLE INPUT AND OUTPUT:

Enter the value

4

Do you want to add another node? Type Yes/No

Yes

Enter the value

1

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

Yes

Enter the value

1

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 4 1 8 1

Enter the element to be deleted

8

The elements in the linked list after deleting the element are 4 1 1

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
struct node {  
    int data;  
    struct node *link;
```

```
};
```

```
void append(struct node **, int);
```

```
void display(struct node *);
```

```
void delete(struct node **, int);
```

```
int main() {
```

```
    struct node *p = NULL;
```

```
    int n;
```

```
    char ch[10];
```

```
    do {
```

```
        printf("Enter the value\n");
```

```
        scanf("%d", &n);
```

```
        append(&p, n);
```

```
        printf("Do you want to add another node? Type Yes/No\n");
```

```
        scanf("%s", ch);
```

```
    } while (!strcmp(ch, "Yes"));
```

```
    printf("The elements in the linked list are\n ");
```

```
    display(p);
```

```
    printf("\n");
```

```
    printf("Enter the element to be deleted\n");
```

```
    scanf("%d", &n);
```

```
    delete(&p, n);
```

```
    printf("The elements in the linked list after deleting the element are ");
```

```
    display(p);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

```

void append(struct node **q, int num) {
    struct node *nn = (struct node *)malloc(sizeof(struct node));
    nn->data = num;
    nn->link = NULL;

    if (*q == NULL) {
        *q = nn;
    } else {
        struct node *temp = *q;
        while (temp->link != NULL)
            temp = temp->link;
        temp->link = nn;
    }
}

```

```

void display(struct node *q) {
    while (q != NULL) {
        printf("%d ", q->data);
        q = q->link;
    }
}

```

```

void delete(struct node **q, int num) {
    struct node *temp = *q;
    struct node *prev = NULL;

    while (temp != NULL) {
        if (temp->data == num) {
            if (prev == NULL)
                *q = temp->link;
            else
                prev->link = temp->link;
        }
        prev = temp;
        temp = temp->link;
    }
}

```

```

        free(temp);
        return; // found return normally
    }
    prev = temp;
    temp = temp->link;
}

// CRT KEY LINE
printf("Element %d is not found in the list\n", num);
exit(0); // STOP program no extra output
}

```

7. Linked List – Reverse

Write a C program to reverse a linked list.

SAMPLE INPUT AND OUTPUT:

```

Enter the value
5
Do you want to add another node? Type Yes/No
Yes
Enter the value
9
Do you want to add another node? Type Yes/No
Yes
Enter the value
2
Do you want to add another node? Type Yes/No
Yes
Enter the value
8
Do you want to add another node? Type Yes/No
No

```

The elements in the linked list are 5 9 2 8

The elements in the reversed linked list are 8 2 9 5

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Node structure
struct node {
    int data;
    struct node *link;
};

// Function declarations
void append(struct node **, int);
void display(struct node *);
void reverse(struct node **);

int main() {
    struct node *head = NULL;
    int value;
    char ch[10];

    // Build the linked list
    do {
        printf("Enter the value\n");
        scanf("%d", &value);
        append(&head, value);

        printf("Do you want to add another node? Type Yes/No\n");
        scanf("%s", ch);
    } while (strcmp(ch, "Yes") == 0);
```

```
printf("The elements in the linked list are ");
display(head);

// Reverse the linked list
reverse(&head);

printf("The elements in the reversed linked list are ");
display(head);

return 0;
}

// Append node at the end
void append(struct node **head, int num) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = num;
    newNode->link = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        struct node *temp = *head;
        while (temp->link != NULL) {
            temp = temp->link;
        }
        temp->link = newNode;
    }
}

// Display all elements
void display(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) {
```



```

        printf("%d ", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

// Reverse the linked list
void reverse(struct node **head) {
    struct node *prev = NULL;
    struct node *current = *head;
    struct node *next = NULL;

    while (current != NULL) {
        next = current->link; // Store next node
        current->link = prev; // Reverse current node's pointer
        prev = current;      // Move prev forward
        current = next;      // Move current forward
    }

    *head = prev; // Update head pointer
}

```

8. Ascending Order Linked List

Write a C program to add a node to an ascending order linked list.

SAMPLE INPUT AND OUTPUT:

Enter the value

5

Do you want to add another node? Type Yes/No

Yes

Enter the value

9

Do you want to add another node? Type Yes/No

Yes

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 2 5 8 9

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Node structure
```

```
struct node {  
    int data;  
    struct node *link;  
};
```

```
// Function declarations
```

```
void insert(struct node **, int);
```

```
void display(struct node *);
```

```

int main() {
    struct node *head = NULL;
    int value;
    char ch[10];

    // Insert nodes
    do {
        printf("Enter the value\n");
        scanf("%d", &value);
        insert(&head, value);

        printf("Do you want to add another node? Type Yes/No\n");
        scanf("%s", ch);
    } while (strcmp(ch, "Yes") == 0);

    printf("The elements in the linked list are ");
    display(head);

    return 0;
}

// Insert node in ascending order
void insert(struct node **head, int num) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = num;
    newNode->link = NULL;

    // If list is empty or new node should be first
    if (*head == NULL || (*head)->data >= num) {
        newNode->link = *head;
        *head = newNode;
        return;
    }
}

```

```

// Find the node after which new node is to be inserted
struct node *temp = *head;
while (temp->link != NULL && temp->link->data < num) {
    temp = temp->link;
}

newNode->link = temp->link;
temp->link = newNode;
}

// Display all elements
void display(struct node *head) {
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

```

9.Doubly Linked List : Append

Write a C program to append a node to the Doubly Linked List.

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to add another node? Type Yes/No

Yes

Enter the value

17

Do you want to add another node? Type Yes/No

Yes

Enter the value

11

Do you want to add another node? Type Yes/No

Yes

Enter the value

28

Do you want to add another node? Type Yes/No

No

The elements in the doubly linked list are 10 17 11 28

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Define a doubly linked list node
```

```
struct dnode {
```

```
    int data;
```

```
    struct dnode *prev;
```

```
    struct dnode *next;
```

```
};
```

```
// Function declarations
```

```
void d_append(struct dnode **head, int num);
```

```
void d_display(struct dnode *head);
```

```
int main() {
```

```
    struct dnode *head = NULL;
```

```
    int value;
```

```
    char ch[10];
```

```

// Append nodes
do {
    printf("Enter the value\n");
    scanf("%d", &value);
    d_append(&head, value);

    printf("Do you want to add another node? Type Yes/No\n");
    scanf("%s", ch);
} while (strcmp(ch, "Yes") == 0);

printf("The elements in the doubly linked list are ");
d_display(head);

return 0;
}

// Append a node at the end of the doubly linked list
void d_append(struct dnode **head, int num) {
    struct dnode *newNode = (struct dnode *)malloc(sizeof(struct dnode));
    newNode->data = num;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (*head == NULL) {
        // If list is empty
        *head = newNode;
    } else {
        struct dnode *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

```

```
}

// Display all elements of the doubly linked list
void d_display(struct dnode *head) {
    struct dnode *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

10. Doubly Linked List : Count

Write a C program to count the number of nodes in the Doubly Linked List.

SAMPLE INPUT AND OUTPUT:

Enter the value

10

Do you want to add another node? Type Yes/No

Yes

Enter the value

17

Do you want to add another node? Type Yes/No

Y

Enter the value

11

Do you want to add another node? Type Yes/No

Y

Enter the value

28

Do you want to add another node? Type Yes/No

N

The elements in the doubly linked list are 10 17 11 28

The number of nodes in the doubly linked list is 4

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Define a doubly linked list node
```

```
struct dnode {
```

```
    int data;
```

```
    struct dnode *prev;
```

```
    struct dnode *next;
```

```
};
```

```
// Function declarations
```

```
void d_append(struct dnode **head, int num);
```

```
void d_display(struct dnode *head);
```

```
int d_count(struct dnode *head);
```

```
int main() {
```

```
    struct dnode *head = NULL;
```

```
    int value;
```

```
    char ch[10];
```

```
    // Append nodes
```



```

do {
    printf("Enter the value\n");
    scanf("%d", &value);
    d_append(&head, value);

    printf("Do you want to add another node? Type Yes/No\n");
    scanf("%s", ch);
} while (strcmp(ch, "Yes") == 0 || strcmp(ch, "Y") == 0);

printf("The elements in the doubly linked list are ");
d_display(head);

printf("The number of nodes in the doubly linked list is %d\n", d_count(head));

return 0;
}

// Append a node at the end of the doubly linked list
void d_append(struct dnode **head, int num) {
    struct dnode *newNode = (struct dnode *)malloc(sizeof(struct dnode));
    newNode->data = num;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        struct dnode *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

```

```
}
```

```
// Display all elements of the doubly linked list
```

```
void d_display(struct dnode *head) {
```

```
    struct dnode *temp = head;
```

```
    while (temp != NULL) {
```

```
        printf("%d ", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
// Count the number of nodes in the doubly linked list
```

```
int d_count(struct dnode *head) {
```

```
    int count = 0;
```

```
    struct dnode *temp = head;
```

```
    while (temp != NULL) {
```

```
        count++;
```

```
        temp = temp->next;
```

```
    }
```

```
    return count;
```

```
}
```

11. Linked List : Find Middle Element

Write a C program to find the middle element in a linked list.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct node
{
    int data;
    struct node* link;
};
void append(struct node** head,int num);
void display(struct node* head);
int count(struct node *head);
int findmid(struct node* head);
int main()
{
    int n;
    char ch[10];
    struct node* p=NULL;
    do
    {
        printf("Enter the value\n");
        scanf("%d",&n);
        append(&p,n);
        printf("Do you want to add another node? Type Yes/No\n");
        scanf("%s",ch);
    }while(!strcmp(ch,"Yes"));
    printf("The elements in the linked list are");
    display(p);
    printf("\nThe middle element in the linked list is %d",findmid(p));
    return 0;
```

```

}
void append(struct node** head,int num)
{
    struct node*newnode=(struct node*)malloc(sizeof(struct node));
    struct node*cur=*head;
    newnode->data=num;
    newnode->link=NULL;
    if(*head==NULL)
    {
        *head=newnode;

    }
    else
    {
        while(cur->link!=NULL)
        {
            cur=cur->link;
        }
        cur->link=newnode;
    }
}

```

```

void display(struct node* head)
{
    //Fill the code here
    while(head!=NULL){
        printf("%d ",head->data);
        head=head->link;
    }
}

```

```

int findmid(struct node* head)
{
    //Fill the code here

```

```

int n=count(head);
int pos,i;
struct node*temp=head;
if(n%2==0)
pos=n/2;
else
pos=n/2+1;
for(i=1;i<pos;i++){
    temp=temp->link;
}
return temp->data;
}

```

```

int count(struct node* head)
{
    //Fill the code here
    int c=0;
    while(head!=NULL){
        c++;
        head=head->link;
    }
    return c;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the value

5

Do you want to add another node? Type Yes/No

Yes

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

9

Do you want to add another node? Type Yes/No

Yes

Enter the value

4

Do you want to add another node? Type Yes/No

Yes

Enter the value

10

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 5 2 9 4 10

The middle element in the linked list is 9

12. Linked List : Print Lots

Write a C program to print a range of elements in a linked list.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct node {
    int data;
    struct node * link;
};
void append ( struct node **, int ) ;
void display ( struct node * ) ;
int count ( struct node *);
void printlots( struct node *, int, int);

int main() {
    struct node *p ;
    p=NULL;
```

```

int n,l,h;
char ch[10];
do {
    printf("Enter the value\n");
    scanf("%d",&n);
    append(&p,n);
    printf("Do you want to add another node? Type Yes/No\n");
    scanf("%s",ch);
} while(!strcmp(ch,"Yes"));
printf("The elements in the linked list are");
display(p);
printf("\n");
printf("Enter the range of elements to print\n");
scanf("%d",&l);
scanf("%d",&h);
if(h>count(p))
    printf("Invalid Range\n");
else {
    printf("The elements in the range %d to %d are \n",l,h);
    printlots(p,l,h);
    printf("\n");
}
return 0;
}

void append ( struct node **q, int num ) {
    struct node*newnode=(struct node*)malloc(sizeof(struct node));
    struct node*temp=*q;

    newnode->data=num;
    newnode->link=NULL;
    if(*q==NULL)
        *q=newnode;
    else{

```

```
        while(temp->link!=NULL)
            temp=temp->link;
        temp->link=newnode;
    }
}
```

```
void display ( struct node *q ) {
    while(q!=NULL){
        printf("%d ",q->data);
        q=q->link;
    }
}
```

```
int count ( struct node *q ) {
    int c=0;
    while(q!=NULL){
        c++;
        q=q->link;
    }
    return c;
}
```

```
void printlots( struct node *q, int low, int high) {
    int pos=1;
    while(q!=NULL){
        if(pos>=low && pos<=high)
            printf("%d ",q->data);
        q=q->link;
        pos++;
    }
}
```


SAMPLE INPUT AND OUTPUT 1:

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

4

Do you want to add another node? Type Yes/No

Yes

Enter the value

6

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

Yes

Enter the value

0

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 2 4 6 8 0

Enter the range of elements to print

2

4

The elements in the range 2 to 4 are

4 6 8

SAMPLE INPUT AND OUTPUT 2:

Enter the value

2

Do you want to add another node? Type Yes/No

Yes

Enter the value

4

Do you want to add another node? Type Yes/No

Yes

Enter the value

6

Do you want to add another node? Type Yes/No

Yes

Enter the value

8

Do you want to add another node? Type Yes/No

Yes

Enter the value

0

Do you want to add another node? Type Yes/No

No

The elements in the linked list are 2 4 6 8 0

Enter the range of elements to print

2

7

Invalid Range

MODULE 3 – Stack ADT And Applications

1.Stack 1 : Array Implementation : Push and Pop

Write a program to implement push and pop operation on stack and to display the contents of the stack.

PROGRAM:

```
#include<stdio.h>

struct stack {
    int a[5];
    int top;
};

void initstack(struct stack * p);
void push(struct stack * p, int item);
void display(struct stack p);
int pop(struct stack * p);
void printMenu();

int main() {
    struct stack p;
    int data,ch, data1;
    initstack(&p);
    do {
        printMenu();
        printf("Enter your choice\n");
        scanf("%d",&ch);
        switch(ch) {
            case 1:
                printf("Enter the element to be pushed\n");
                scanf("%d",&data);
```

```
    push(&p, data);
    break;
case 2:
    data1 = pop(&p);
    if(data1 != -1000)
        printf("The popped element is %d\n",data1);
    break;
case 3:
    printf("The contents of the stack are");
    display(p);
    printf("\n");
    break;
default:
    return 0;
}
} while(1);
return 0;
}
```

```
void printMenu()
{
    printf("Choice 1 : Push\n");
    printf("Choice 2 : Pop\n");
    printf("Choice 3 : Display\n");
    printf("Any other choice : Exit\n");
}
```

```
void initstack(struct stack * p) {
    p->top = -1;
```

```

    }

void push(struct stack * p, int item) {
    if(p->top == 4) {
        printf("Stack is full\n");
        return;
    }
    p->top++;
    p->a[p->top] = item;
}

void display(struct stack p) {
    int i;
    if(p.top == -1) {
        printf(" {}");
        return;
    }
    for(i = p.top; i >= 0; i--) {
        printf(" %d", p.a[i]);
    }
}

int pop(struct stack * p) {
    int item;
    if(p->top == -1) {
        printf("Stack is empty\n");
        return -1000;
    }
    item = p->a[p->top];
    p->top--;
    return item;
}

```

SAMPLE INPUT AND OUTPUT:

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are 3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

5

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 5

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

Stack is empty

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

2

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

4

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

5

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

6

Stack is full

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are 5 4 3 2 1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 5

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are 4 3 2 1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 4

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are 3 2 1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

5

2.Stack 2 : Array Implementation : Push and Pop

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct stack {
```

```
    int * a;
```

```
    int top;
```

```
    int maxSize;
```

```
};
```

```
void initstack(struct stack * p, int maxSize);

void push(struct stack * p, int item);

void display(struct stack p);

int pop(struct stack * p);

void printMenu();


int main() {

    struct stack p;

    int data,ch, data1, m;

    printf("Enter the maximum size of the stack\n");

    scanf("%d",&m);

    initstack(&p,m);

    do {

        printMenu();

        printf("Enter your choice\n");

        scanf("%d",&ch);

        switch(ch) {

            case 1:

                printf("Enter the element to be pushed\n");

                scanf("%d",&data);

                push(&p, data);

                break;

            case 2:

                data1 = pop(&p);

                if(data1 != -1000)

                    printf("The popped element is %d\n",data1);

                break;

            case 3:

                printf("The contents of the stack are");
```

```
        display(p);
        printf("\n");
        break;
    default:
        return 0;
    }
} while(1);
return 0;
}
```

```
void printMenu()
```

```
{
    printf("Choice 1 : Push\n");
    printf("Choice 2 : Pop\n");
    printf("Choice 3 : Display\n");
    printf("Any other choice : Exit\n");
}
```

```
void initstack(struct stack * p, int maxSize) {
```

```
    p->top = -1;
    p->maxSize = maxSize;
    p->a = (int *)malloc(maxSize * sizeof(int));
}
```

```
void push(struct stack * p, int item) {
```

```
    if(p->top == p->maxSize - 1) {
        printf("Stack is full\n");
        return;
    }
    p->top = p->top + 1;
    *(p->a + p->top) = item;
```

```

    }

void display(struct stack p) {
    int i;if(p.top == -1) {
        printf(" {}");
        return;
    }
    printf(" ");
    for(i = p.top; i >= 0; i--) {
        printf("%d ", *(p.a + i));
    }
}

int pop(struct stack * p) {
    int item;
    if(p->top == -1) {
        printf("Stack is empty\n");
        return -1000;
    }
    item = *(p->a + p->top);
    p->top = p->top - 1;
    return item;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the maximum size of the stack

3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

2

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be pushed

4

Stack is full

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are 3 2 1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 3

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 2

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The popped element is 1

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

Stack is empty

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the stack are {}

Choice 1 : Push

Choice 2 : Pop

Choice 3 : Display

Any other choice : Exit

Enter your choice

4

3.Stack as a Linked List

Write a menu driven program to implement stack as a linked list. Assume that the size of the stack is 5.

PROGRAM:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *link;
```



```
};
```

```
void push(struct node **, int);
```

```
void display(struct node *);
```

```
int count(struct node *);
```

```
void pop(struct node **);
```

```
int main() {
```

```
    struct node *p = NULL;
```

```
    int choice;
```

```
    printf("Choice 1 : To push data\n");
```

```
    printf("Choice 2 : To pop data\n");
```

```
    printf("Choice 3 : To display\n");
```

```
    printf("Choice 4 : To count\n");
```

```
    printf("\n");
```

```
    do {
```

```
        printf("Enter your choice\n");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: {
```

```
                printf("Enter the data to be pushed\n");
```

```
                int d;
```

```
                scanf("%d", &d);
```

```
                push(&p, d);
```

```
                break;
```

```
            }
```

case 2:

pop(&p);

break;

case 3:

printf("Stack Contents are\n");

display(p);

printf("\n");

break;

case 4:

printf("Number of elements in the linked list is %d\n", count(p));

break;

default:

printf("END\n");

return 0;

}

} while (1);

return 0;

}

void push(struct node **p, int ele) {

if (count(*p) == 5) {

printf("Overflow\n");

return;

}

SAMPLE INPUT AND OUTPUT:

Choice 1 : To push data

Choice 2 : To pop data

Choice 3 : To display

Choice 4 : To count

Enter your choice

1

Enter the data to be pushed

34

Enter your choice

1

Enter the data to be pushed

23

Enter your choice

1

Enter the data to be pushed

55

Enter your choice

3

Stack Contents are

55 23 34

Enter your choice

1

Enter the data to be pushed

89

Enter your choice

3

Stack Contents are

89 55 23 34

Enter your choice

4

Number of elements in the linked list is 4

Enter your choice

2

Element Popped is 89

Enter your choice

3

Stack Contents are

55 23 34

Enter your choice

2
Element Popped is 55
Enter your choice
2
Element Popped is 23
Enter your choice
4
Number of elements in the linked list is 1
Enter your choice
2
Element Popped is 34
Enter your choice
2
There is no elements in the stack
Enter your choice
4
Number of elements in the linked list is 0
Enter your choice
3
Stack Contents are
{}
Enter your choice
6
END

4.Parentheses Balance

You are given a string consisting of parentheses () and []. A string of this type is said to be *correct*:

- (a) if it is the empty string
- (b) if A and B are correct, AB is correct,
- (c) if A is correct, (A) and [A] is correct.

Write a program that takes a sequence of strings of this type and check their correctness. Your program can assume that the maximum string length is 128.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>
```

```
struct stack
```

```
{
```

```
    char *s;
```

```
    int top;
```

```
};
```

```
void initstack(struct stack * p){
```

```
    p->top = -1;
```

```
    p->s = (char *)malloc(128 * sizeof(char));
```

```
}
```

```
void push(struct stack *p,char c){
```

```
    p->top++;
```

```
    p->s[p->top] = c;
```

```
}
```

```
int isEmpty(struct stack p){
```

```
    if (p.top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
char pop(struct stack *p){
```

```
    if (p->top == -1)
```

```
        return '\0';
```

```
    return p->s[p->top--];
```

```
    return 0;
```

```
}
```

```
void balancedornot(struct stack p, char *par){
```

```
    initstack(&p);
```

```
for (int i = 0; par[i] != '\0'; i++) {  
    char ch = par[i];  
    if (ch == '(' || ch == '[') {  
        push(&p, ch);  
    }  
    else if (ch == ')' || ch == ']') {  
        if (isEmpty(p)) {  
            printf("No\n");  
            return;  
        }  
        char top = pop(&p);  
        if ((ch == ')' && top != '(') ||  
            (ch == ']' && top != '[')) {  
            printf("No\n");  
            return;  
        }  
    }  
}  
  
if (isEmpty(p))  
    printf("Yes\n");  
else  
    printf("No\n");  
}
```

```
int main(){  
    int n;  
    struct stack p;  
    char par[50];  
    scanf("%d", &n);
```

```

    getchar();
    while (n--){
        scanf("%s",par);
        balancedornot(p,par);
    }
    return 0;
}

```

SAMPLE INPUT:

```

3
([
(([])))
([()()])

```

SAMPLE OUTPUT:

```

Yes
No
Yes

```

5.Equation - Infix to Postfix Conversion

PROGRAM:

```

#include<stdio.h>
#include<stdlib.h>

struct node {
    int data;
    struct node *link;
};

void push(struct node **, int);

```

```
void display(struct node *);  
  
int count(struct node *);  
  
void pop(struct node **);  
  
  
int main() {  
    struct node *p = NULL;  
    int choice;  
  
    printf("Choice 1 : To push data\n");  
    printf("Choice 2 : To pop data\n");  
    printf("Choice 3 : To display\n");  
    printf("Choice 4 : To count\n");  
    printf("\n");  
  
    do {  
        printf("Enter your choice\n");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1: {  
                printf("Enter the data to be pushed\n");  
                int d;  
                scanf("%d", &d);  
                push(&p, d);  
                break;  
            }  
  
            case 2:  
                pop(&p);  

```



```
break;
```

```
case 3:
```

```
printf("Stack Contents are\n");
```

```
display(p);
```

```
printf("\n");
```

```
break;
```

```
case 4:
```

```
printf("Number of elements in the linked list is %d\n", count(p));
```

```
break;
```

```
default:
```

```
printf("END\n");
```

```
return 0;
```

```
}
```

```
} while (1);
```

```
return 0;
```

```
}
```

```
void push(struct node **p, int ele) {
```

```
if (count(*p) == 5) {
```

```
printf("Overflow\n");
```

```
return;
```

```
}
```

SAMPLE INPUT AND OUTPUT:

Enter an expression in infix form:

(a*b+d)

The postfix expression is:

ab*d+

6.Evaluate an expression entered in postfix form

Write a program to evaluate an expression entered in “postfix” form using stack concept.

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#include<ctype.h>
```

```
int MAX = 50;
```

```
struct postfix {
    int stack[50];
    int top, nn;
    char *s;
};
```

```
void initpostfix(struct postfix *);
void setexpr(struct postfix *, char *str);
void push(struct postfix *, int item);
int pop(struct postfix *);
void calculate(struct postfix *);
```

```
void show(struct postfix);
```

```
int main() {
```

```
    char expr[50];
```

```
    printf("Enter postfix expression to be evaluated:\n");
```

```
    scanf("%s", expr);
```

```
    struct postfix p;
```

```
    initpostfix(&p);
```

```
    setexpr(&p, expr);
```

```
    calculate(&p);
```

```
    show(p);
```

```
    return 0;
```

```
}
```

```
void initpostfix(struct postfix *p) {
```

```
    p->top = -1;
```

```
}
```

```
void setexpr(struct postfix *p, char *str) {
```

```
    p->s = str;
```

```
}
```

```
void push(struct postfix *p, int item) {
```

```
    p->top = p->top + 1;
```

```
    p->stack[p->top] = item;
```

```
}
```

```
int pop(struct postfix *p) {  
    int item;  
    item = p->stack[p->top];  
    p->top = p->top - 1;  
    return item;  
}
```

```
void calculate(struct postfix *p) {  
    int op1, op2, res;  
    char *c = p->s;  
  
    while (*c) {  
        if (isdigit(*c)) {  
            push(p, *c - '0');  
        } else {  
            op2 = pop(p);  
            op1 = pop(p);  
  
            switch (*c) {  
                case '+': res = op1 + op2; break;  
                case '-': res = op1 - op2; break;  
                case '*': res = op1 * op2; break;  
                case '/': res = op1 / op2; break;  
            }  
  
            push(p, res);  
        }  
    }
```

```

        c++;
    }

    p->nn = p->stack[p->top];
}

void show(struct postfix p) {
    printf("Result is: %d\n", p.nn);
}

```

SAMPLE INPUT AND OUTPUT:

Enter postfix expression to be evaluated:

13+

Result is: 4

7.Iterative Tower of Hanoi

PROGRAM:

```

#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

#include <math.h>


// Structure for stack
struct stack {
    int *a;
    int top;
    int maxSize;
};


// Function declarations

```

```
void initstack(struct stack *p, int maxSize);
void push(struct stack *p, int item);
int pop(struct stack *p);
void moveDisk(char fromPeg, char toPeg, int disk);
void moveDisksBetweenTwoPoles(struct stack *src, struct stack *dest, char s, char d);
void hanoiIterative(int num_of_disks, struct stack *src, struct stack *aux, struct stack *dest);
```

```
int main() {
    int num_of_disks;

    printf("Enter the number of disks\n");
    scanf("%d", &num_of_disks);

    struct stack src, dest, aux;

    initstack(&src, num_of_disks);
    initstack(&aux, num_of_disks);
    initstack(&dest, num_of_disks);

    // Push disks into source pole (largest at bottom)
    for (int i = num_of_disks; i >= 1; i--) {
        push(&src, i);
    }

    hanoiIterative(num_of_disks, &src, &aux, &dest);

    return 0;
}

// Initialize stack
```

```
void initstack(struct stack *p, int maxSize) {  
    p->top = -1;  
    p->maxSize = maxSize;  
    p->a = (int *)malloc(maxSize * sizeof(int));  
}
```

// Push element

```
void push(struct stack *p, int item) {  
    if (p->top == p->maxSize - 1)  
        return;  
    p->a[++p->top] = item;  
}
```

// Pop element

```
int pop(struct stack *p) {  
    if (p->top == -1)  
        return INT_MIN;  
    return p->a[p->top--];  
}
```

// Move disk

```
void moveDisk(char fromPeg, char toPeg, int disk) {  
    printf("Move the disk %d from %c to %c\n", disk, fromPeg, toPeg);  
}
```

// Move disks between poles

```
void moveDisksBetweenTwoPoles(struct stack *src, struct stack *dest, char s, char d) {  
    int pole1TopDisk = pop(src);  
    int pole2TopDisk = pop(dest);
```

```
// When src pole is empty
if (pole1TopDisk == INT_MIN) {
    push(src, pole2TopDisk);
    moveDisk(d, s, pole2TopDisk);
}

// When dest pole is empty
else if (pole2TopDisk == INT_MIN) {
    push(dest, pole1TopDisk);
    moveDisk(s, d, pole1TopDisk);
}

// When top disk of src > dest
else if (pole1TopDisk > pole2TopDisk) {
    push(src, pole1TopDisk);
    push(src, pole2TopDisk);
    moveDisk(d, s, pole2TopDisk);
}

// When top disk of src < dest
else {
    push(dest, pole2TopDisk);
    push(dest, pole1TopDisk);
    moveDisk(s, d, pole1TopDisk);
}

// Iterative Tower of Hanoi
```



```

void hanoiIterative(int num_of_disks, struct stack *src, struct stack *aux, struct stack *dest) {
    int total_moves;
    char s = 'S', a = 'A', d = 'D';

    // If number of disks is even, swap destination and auxiliary
    if (num_of_disks % 2 == 0) {
        char temp = d;
        d = a;
        a = temp;
    }

    total_moves = pow(2, num_of_disks) - 1;

    for (int i = 1; i <= total_moves; i++) {
        if (i % 3 == 1)
            moveDisksBetweenTwoPoles(src, dest, s, d);
        else if (i % 3 == 2)
            moveDisksBetweenTwoPoles(src, aux, s, a);
        else
            moveDisksBetweenTwoPoles(aux, dest, a, d);
    }
}

```

SAMPLE INPUT AND OUTPUT:

Enter the number of disks

3

Move the disk 1 from 'S' to 'D'
 Move the disk 2 from 'S' to 'A'
 Move the disk 1 from 'D' to 'A'
 Move the disk 3 from 'S' to 'D'
 Move the disk 1 from 'A' to 'S'
 Move the disk 2 from 'A' to 'D'
 Move the disk 1 from 'S'

MODULE 4 – Queue ADT And Applications

1.Queue : Array Implementation using element count

Write a program to implement enQueue and deQueue operation on queue and to display the contents of the queue.

PROGRAM:

```
#include<stdio.h>

#include<string.h>

struct queue {

    int contents[5];

    int front;

    int count;

} ;

void printMenu(){

printf("Choice 1 : Enter element into Queue\n");

printf("Choice 2 : Delete element from Queue\n");

printf("Choice 3 : Display\n");

printf("Any other choice : Exit\n");

}void initQueue(struct queue * q){

q->front = 0;

q->count = 0;

}

void enQueue(struct queue * q, int element){

if(q->count == 5) {

printf("Queue is full\n");

return;

}

int rear = (q->front + q->count) % 5;

q->contents[rear] = element;
```

```

q->count++;
}

int deQueue(struct queue * q){
if(q->count == 0) {
printf("Queue is empty\n");
return -1000;
}

int ele = q->contents[q->front];
q->front = (q->front + 1) % 5;
q->count--;
return ele;
return 0;
}

void display( struct queue q){
if(q.count == 0) {
printf(" {}");
return;
}for(int i = 0; i < q.count; i++) {
int index = (q.front + i) % 5;
printf(" %d", q.contents[index]);
}}

int main()
{
    struct queue p;
    int data,ch, data1;
    initQueue(&p);
    do {
        printMenu();
        printf("Enter your choice\n");

```

```
scanf("%d",&ch);
switch(ch) {
    case 1:
        printf("Enter the element to be inserted/entered\n");
        scanf("%d",&data);
        enqueue(&p, data);
        break;
    case 2:
        data1 = dequeue(&p);
        if(data1 != -1000)
            printf("The deleted element is %d\n",data1);
        break;
    case 3:
        printf("The contents of the queue are");
        display(p);
        printf("\n");
        break;
    default:
        return 0;
}
} while(1);
return 0;
}
```

SAMPLE INPUT AND OUTPUT:

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are {}

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

10

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

20

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

30

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

40

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

50

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

60

Queue is full

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 10

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 20

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

60

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are 30 40 50 60

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 30

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 40

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 50

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are 60

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 60

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

Queue is empty

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are {}

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

4

2.Queue - Linked List Implementation

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct node {
```

```
    int data;
```

```
    struct node* next;
```

```
} Node;
```

```

/* function to insert a node in a circular queue */

void addcirq(struct node **f, struct node **r, int item) {

    struct node *newnode = (struct node *)malloc(sizeof(struct node));

    newnode->data = item;

    if (*f == NULL) {

        *f = *r = newnode;

        newnode->next = newnode; // circular link

    } else {

        newnode->next = *f;

        (*r)->next = newnode;

        *r = newnode;

    }

}

```

```

/* function to delete a node in a circular queue */

int delcirq(struct node **f, struct node **r) {

    if (*f == NULL) {

        printf("Queue is empty\n");

        return -1000;

    }

    int item;

    struct node *temp = *f;

    if (*f == *r) { // only one node

        item = temp->data;

        *f = *r = NULL;
    }
}

```

```

    } else {
        item = temp->data;
        *f = temp->next;
        (*r)->next = *f;
    }

    free(temp);
    return item;
}

/* function to display circular queue */
void cirq_display(struct node *f) {
    if (f == NULL) {
        printf(" {}");
        return;
    }

    struct node *temp = f;
    do {
        printf(" %d", temp->data);
        temp = temp->next;
    } while (temp != f);
}

int main() {
    int data, ch, data1;
    Node *front = NULL, *rear = NULL;

    do {

```

```
printf("Choice 1 : Enter element into Queue\n");
printf("Choice 2 : Delete element from Queue\n");
printf("Choice 3 : Display\n");
printf("Any other choice : Exit\n");
printf("Enter your choice\n");
scanf("%d", &ch);

switch (ch) {
    case 1:
        printf("Enter the element to be inserted/entered\n");
        scanf("%d", &data);
        addcirq(&front, &rear, data);
        break;

    case 2:
        data1 = delcirq(&front, &rear);
        if (data1 != -1000)
            printf("The deleted element is %d\n", data1);
        break;

    case 3:
        printf("The contents of the queue are");
        cirq_display(front);
        printf("\n");
        break;
}

} while (ch > 0 && ch < 4);
```

```
    return 0;  
}
```

SAMPLE INPUT AND OUTPUT:

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

20

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

30

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

40

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are 20 30 40

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

1

Enter the element to be inserted/entered

50

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are 20 30 40 50

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 20

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 30

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 40

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

The deleted element is 50

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

2

Queue is empty

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

3

The contents of the queue are {}

Choice 1 : Enter element into Queue

Choice 2 : Delete element from Queue

Choice 3 : Display

Any other choice : Exit

Enter your choice

4

3. Deque

Write a menu driven C program to maintain a Double Ended queue / Deque using an array.

PROGRAM:

```
#include <stdio.h>
```

```
#define SIZE 10
```

```
void addqatbeg(int deque[], int data, int *front, int *rear);
```

```
void addqatend(int deque[], int data, int *front, int *rear);
```

```
void delqatbeg(int deque[], int *front, int *rear);
```

```
void delqatend(int deque[], int *front, int *rear);
```

```
void display(int deque[], int front, int rear);
```

```
void count(int deque[], int front, int rear);
```

```
int main() {
```

```
    int deque[SIZE] = {0};
```

```
    int front = -1, rear = -1;
```

```
    int choice, data;
```



```
printf("Choice 1 : To add data at the beginning\n");
printf("Choice 2 : To add data at the end\n");
printf("Choice 3 : To delete data at the beginning\n");
printf("Choice 4 : To delete data at the end\n");
printf("Choice 5 : To display\n");
printf("Choice 6 : To count\n");
```

```
while (1) {
    printf("Enter your choice\n");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter the data to be added at the beginning\n");
            scanf("%d", &data);
            addqatbeg(deque, data, &front, &rear);
            break;

        case 2:
            printf("Enter the data to be added at the end\n");
            scanf("%d", &data);
            addqatend(deque, data, &front, &rear);
            break;

        case 3:
            delqatbeg(deque, &front, &rear);
            break;
```

case 4:

delqatend(deque, &front, &rear);

break;

case 5:

display(deque, front, rear);

break;

case 6:

count(deque, front, rear);

break;

default:

return 0;

}

}

}

/* Add at beginning */

void addqatbeg(int deque[], int data, int *front, int *rear) {

if (*front == 0 && *rear == SIZE - 1) {

printf("Deque is full\n");

return;

}

if (*front == -1) {

*front = *rear = 0;

}

else if (*front > 0) {

```

        (*front)--;
    }
    else {
        for (int i = *rear; i >= *front; i--)
            deque[i + 1] = deque[i];
        (*rear)++;
    }

    deque[*front] = data;
}

/* Add at end */
void addqatend(int deque[], int data, int *front, int *rear) {
    if (*front == 0 && *rear == SIZE - 1) {
        printf("Deque is full\n");
        return;
    }

    if (*front == -1) {
        *front = *rear = 0;
    }
    else if (*rear < SIZE - 1) {
        (*rear)++;
    }
    else {
        for (int i = *front; i <= *rear; i++)
            deque[i - 1] = deque[i];
        (*front)--;
    }
}

```

```
    deque[*rear] = data;
}

/* Delete from beginning */
void delqatbeg(int deque[], int *front, int *rear) {
    if (*front == -1) {
        printf("Deque is empty\n");
        return;
    }

    int item = deque[*front];
    deque[*front] = 0;

    if (*front == *rear)
        *front = *rear = -1;
    else
        (*front)++;

    printf("Item extracted : %d\n", item);
}

/* Delete from end */
void delqatend(int deque[], int *front, int *rear) {
    if (*front == -1) {
        printf("Deque is empty\n");
        return;
    }
}
```

```

int item = deque[*rear];
deque[*rear] = 0;

if (*front == *rear)
    *front = *rear = -1;
else
    (*rear)--;

printf("Item extracted : %d\n", item);
}

/* Display deque */
void display(int deque[], int front, int rear) {
    if (front == -1) {
        printf("Deque is empty\n");
        return;
    }

    printf("front: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", deque[i]);
    printf(":rear\n");
}

/* Count elements */
void count(int deque[], int front, int rear) {
    int c;

    if (front == -1)

```

```
        c = 0;

    else

        c = rear - front + 1;


    printf("Total number of elements in deque : %d\n", c);
}
```

SAMPLE INPUT AND OUTPUT:

Choice 1 : To add data at the beginning
Choice 2 : To add data at the end
Choice 3 : To delete data at the beginning
Choice 4 : To delete data at the end
Choice 5 : To display
Choice 6 : To count
Enter your choice
2
Enter the data to be added at the end
17
Enter your choice
1
Enter the data to be added at the beginning
10
Enter your choice
2
Enter the data to be added at the end
8
Enter your choice
1
Enter the data to be added at the beginning
-9
Enter your choice
2
Enter the data to be added at the end
13
Enter your choice
1
Enter the data to be added at the beginning
28
Enter your choice
2

Enter the data to be added at the end

14

Enter your choice

1

Enter the data to be added at the beginning

5

Enter your choice

2

Enter the data to be added at the end

25

Enter your choice

1

Enter the data to be added at the beginning

6

Enter your choice

2

Enter the data to be added at the end

21

Deque is full

Enter your choice

1

Enter the data to be added at the beginning

11

Deque is full

Enter your choice

5

front: 6 5 28 -9 10 17 8 13 14 25 :rear

Enter your choice

6

Total number of elements in deque : 10

Enter your choice

3

Item extracted : 6

Enter your choice

3

Item extracted : 5

Enter your choice

3

Item extracted : 28

Enter your choice

5

front: -9 10 17 8 13 14 25 :rear

Enter your choice

2

Enter the data to be added at the end

16

Enter your choice

2

Enter the data to be added at the end

7

Enter your choice

5

front: -9 10 17 8 13 14 25 16 7 :rear

Enter your choice

4

Item extracted : 7

Enter your choice

4

Item extracted : 16

Enter your choice

5

front: -9 10 17 8 13 14 25 :rear

Enter your choice

6

Total number of elements in deque : 7

Enter your choice

7

4.Ferry Loading – III

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 10005
```

```
typedef struct {
```

```
    int time;
```

```
    int idx;
```

```
} Car;
```

```
Car leftQ[MAX], rightQ[MAX];
```

```
int lf = 0, lr = 0, rf = 0, rr = 0;
```



```
void enqueue(Car q[], int *r, Car c) {  
    q[(*r)++] = c;  
}
```

```
Car dequeue(Car q[], int *f) {  
    return q[(*f)++];  
}
```

```
Car peek(Car q[], int f) {  
    return q[f];  
}
```

```
int isEmpty(int f, int r) {  
    return f == r;  
}
```

```
int main() {  
    int n, t, m;  
    scanf("%d", &n);  
    scanf("%d", &t);  
    scanf("%d", &m);  
  
    int ans[MAX];  
    char bank[10];  
    int time;  
  
    for (int i = 0; i < m; i++) {  
        scanf("%d %s", &time, bank);  
        Car c = {time, i};
```

```

    if (strcmp(bank, "left") == 0)
        enqueue(leftQ, &lr, c);
    else
        enqueue(rightQ, &rr, c);
}

int curTime = 0;
int onLeft = 1; // ferry starts at left

while (!isEmpty(lf, lr) || !isEmpty(rf, rr)) {
    int nextArrival = 1000000000;

    if (!isEmpty(lf, lr))
        nextArrival = leftQ[lf].time;

    if (!isEmpty(rf, rr) && rightQ[rf].time < nextArrival)
        nextArrival = rightQ[rf].time;

    if (curTime < nextArrival)
        curTime = nextArrival;

    int cnt = 0;

    if (onLeft) {
        while (!isEmpty(lf, lr) && leftQ[lf].time <= curTime && cnt < n) {
            Car c = dequeue(leftQ, &lf);
            ans[c.idx] = curTime + t;
            cnt++;
        }
    }
}

```

```

    }
} else {
    while (!isEmpty(rf, rr) && rightQ[rf].time <= curTime && cnt < n) {
        Car c = dequeue(rightQ, &rf);
        ans[c.idx] = curTime + t;
        cnt++;
    }
}

curTime += t;
onLeft = !onLeft;
}

printf("The unloaded time\n");
for (int i = 0; i < m; i++) {
    printf("%d\n", ans[i]);
}

return 0;
}

```

SAMPLE INPUT AND OUTPUT 1:

```

2
10
10
0 left
10 left
20 left
30 left
40 left
50 left
60 left
70 left

```

80 left
90 left
The unloaded time
10
30
30
50
50
70
70
90
90
110

SAMPLE INPUT AND OUTPUT 2:

3
5
7
0 left
1 left
2 left
3 left
5 left
5 right
11 right
The unloaded time
5
15
15
15
25
10
20

5.Ferry Loading – IV

PROGRAM:

```
#include <stdio.h>

#include <string.h>

#define MAX 10005
```

```
int leftQ[MAX], rightQ[MAX];
```

```
int lf = 0, lr = 0, rf = 0, rr = 0;
```

```
void enqueue(int q[], int *r, int val) {
```

```
    q[(*r)++] = val;
```

```
}
```

```
int dequeue(int q[], int *f) {
```

```
    return q[(*f)++];
```

```
}
```

```
int isEmpty(int f, int r) {
```

```
    return f == r;
```

```
}
```

```
int main() {
```

```
    int L, m;
```

```
    scanf("%d", &L);
```

```
    scanf("%d", &m);
```

```
    int ferryLen = L * 100; // convert to cm
```

```
    int len;
```

```
    char bank[10];
```

```
    for (int i = 0; i < m; i++) {
```

```
        scanf("%d %s", &len, bank);
```

```
        if (strcmp(bank, "left") == 0)
```

```
            enqueue(leftQ, &lr, len);
```

```

        else
            enqueue(rightQ, &rr, len);
    }

    int count = 0;
    int onLeft = 1; // ferry starts at left

    while (!isEmpty(lf, lr) || !isEmpty(rf, rr)) {
        int curLen = 0;

        if (onLeft) {
            while (!isEmpty(lf, lr) && curLen + leftQ[lf] <= ferryLen) {
                curLen += dequeue(leftQ, &lf);
            }
        } else {
            while (!isEmpty(rf, rr) && curLen + rightQ[rf] <= ferryLen) {
                curLen += dequeue(rightQ, &rf);
            }
        }

        count++;
        onLeft = !onLeft; // cross river
    }

    printf("The number of times the ferry has to cross the river\n");
    printf("%d\n", count);

    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

20
4
380 left
720 left
1340 right
1040 left
The number of times the ferry has to cross the river
3

6.Shankar and his new Project**PROGRAM:**

```
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    int calling[100], ideal[100];

    for (int i = 0; i < N; i++)
        scanf("%d", &calling[i]);

    for (int i = 0; i < N; i++)
        scanf("%d", &ideal[i]);

    int time = 0;
    int front = 0;
    int idealIndex = 0;
    int size = N;

    while (idealIndex < N) {
        if (calling[front] == ideal[idealIndex]) {
```

```

        // Execute process
        time++;
        front = (front + 1) % N;
        idealIndex++;
        size--;
    } else {
        // Move front element to end
        int temp = calling[front];
        front = (front + 1) % N;
        calling[(front + size - 1) % N] = temp;
        time++;
    }
}

printf("%d\n", time);

return 0;
}

```

SAMPLE INPUT AND OUTPUT:

3

3 2 1

1 3 2

5

MODULE 5 – Binary Trees And Traversals

1.Complete Binary Tree and Traversals

Implement a C program to construct a Complete Binary Tree and also display the elements in the tree using Inorder , Preorder and Postorder traversals.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct tree {
    int data;

    struct tree *left, *right;
};

struct queue {
    int front, rear, size;
    struct tree **array;
};

struct tree* newNode(int data) {
    struct tree* temp = (struct tree*)malloc(sizeof(struct tree));
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}

struct queue* createQueue(int size) {
    struct queue* q = (struct queue*)malloc(sizeof(struct queue));
    q->front = q->rear = -1;
    q->size = size;

    q->array = (struct tree**)malloc(size * sizeof(struct tree*));
    return q;
}

void enqueue(struct tree* node, struct queue* q) {
    if (q->rear == -1) {
```

```

        q->front = q->rear = 0;

    } else {

        q->rear++;

    }

    q->array[q->rear] = node;
}

struct tree* dequeue(struct queue* q) {
    return q->array[++q->front];
}

void insert(struct tree **root, int data, struct queue* q) {
    struct tree *temp = newNode(data);
    if (*root == NULL) {

        *root = temp;
        enqueue(*root, q);
        return;
    }

    struct tree *frontNode = q->array[q->front + 1];
    if (frontNode->left == NULL) {
        frontNode->left = temp;
        enqueue(temp, q);
    }

    else if (frontNode->right == NULL) {
        frontNode->right = temp;
        enqueue(temp, q);
        dequeue(q);
    }

}

void inorder(struct tree *root) {
    if (root != NULL) {
        inorder(root->left);

```

```

        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct tree *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct tree *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct tree *root = NULL;

    struct queue *q = createQueue(100);
    char choice[5];
    int data;
    do {
        printf("Enter the element to be inserted in the tree\n");
        scanf("%d", &data);
        insert(&root, data, q);

        printf("Do you want to insert another element?\n");
        scanf("%s", choice);
    } while (strcmp(choice, "yes") == 0);

    printf("Inorder Traversal : The elements in the tree are "); inorder(root);
    printf("\nPreorder Traversal : The elements in the tree are ");
    preorder(root);
    printf("\nPostorder Traversal : The elements in the tree are ");
    postorder(root);
    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the element to be inserted in the tree

1

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

2

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

3

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

4

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

5

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

6

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

7

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

8

Do you want to insert another element?

no

Inorder Traversal : The elements in the tree are 8 4 2 5 1 6 3 7

Preorder Traversal : The elements in the tree are 1 2 4 8 5 3 6 7

Postorder Traversal : The elements in the tree are 8 4 5 2 6 7 3 1

2.Implementation of expression tree and traversals

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
/* Tree structure */
struct tree {
    char data;
    struct tree *left;
    struct tree *right;
};

/* Stack implementation */
struct tree *stack[100];
int top = -1;

/* Create new node */
struct tree* newnode(char b) {
    struct tree* temp = (struct tree*)malloc(sizeof(struct tree));
    temp->data = b;
    temp->left = temp->right = NULL;
    return temp;
}

/* Push into stack */
void push(struct tree *temp) {
    stack[++top] = temp;
}

/* Pop from stack */
struct tree* pop() {
    return stack[top--];
}

/* Inorder Traversal */
void inorder(struct tree *t) {
    if (t != NULL) {
        inorder(t->left);
        printf(" %c",t->data);
        inorder(t->right);
    }
}
```

```
/* Preorder Traversal */
```

```
void preorder(struct tree *t) {  
    if (t != NULL) {  
        printf(" %c",t->data);  
        preorder(t->left);  
        preorder(t->right);  
    }  
}
```

```
/* Postorder Traversal */
```

```
void postorder(struct tree *t) {  
    if (t != NULL) {  
        postorder(t->left);  
        postorder(t->right);  
        printf(" %c",t->data);  
    }  
}
```

```
/* Main function */
```

```
int main() {  
    char postfix[100];  
    int i;  
    printf("Enter the postfix expression:\n");  
    scanf("%s", postfix);  
    for (i = 0; postfix[i] != '\0'; i++) {  
  
        // Operand  
  
        if (isdigit(postfix[i])) {  
            push(newnode(postfix[i]));  
        }  
  
        // Operator  
        else {  
            struct tree *t = newnode(postfix[i]);  
            t->right = pop();  
            t->left = pop();  
            push(t);  
        }  
    }  
}
```

```
struct tree *root = pop();  
printf("\nInorder Traversal\n");  
inorder(root);
```

```

printf("\nPreorder Traversal\n");
preorder(root);
printf("\nPostorder Traversal\n");
postorder(root);
return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Sample Input and Output:

Enter the postfix expression:

23*45/56+-*

Inorder Traversal

2 * 3 * 4 / 5 - 5 + 6

Preorder Traversal

* * 2 3 - / 4 5 + 5 6

Postorder Traversal

2 3 * 4 5 / 5 6 + - *

3.Binary Search Tree

PROGRAM:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct tnode
{
    int data;

    struct tnode *leftc;
    struct tnode *rightc;
};

/* Function to insert an element into BST */
void insert(struct tnode **root, int num)
{
    if (*root == NULL)
    {
        *root = (struct tnode*)malloc(sizeof(struct tnode));
        (*root)->data = num;
        (*root)->leftc = NULL;
    }
}

```

```
    (*root)->rightc = NULL;
    return;
}
```

```
if (num < (*root)->data)
    insert(&((*root)->leftc), num);
else
    insert(&((*root)->rightc), num);
}
```

```
/* Inorder Traversal */
```

```
void inorder(struct tnode *s)
```

```
{
    if (s != NULL)
    {
        inorder(s->leftc);
        printf(" %d", s->data);
        inorder(s->rightc);
    }
}
```

```
/* Preorder Traversal */
```

```
void preorder(struct tnode *s)
```

```
{
    if (s != NULL)
    {
        printf(" %d", s->data);
        preorder(s->leftc);
        preorder(s->rightc);
    }
}
```

```
/* Postorder Traversal */
```

```
void postorder(struct tnode *s)
```

```
{
```



```

    if (s != NULL)
    {
        postorder(s->leftc);
        postorder(s->rightc);

        printf("%d ", s->data);
    }
}

/* Main function */

int main()
{
    struct tnode *root = NULL;
    int num;
    char choice[10];
    do
    {
        printf("Enter the element to be inserted in the tree\n");
        scanf("%d", &num);
        insert(&root, num);

        printf("Do you want to insert another element?\n");
        scanf("%s", choice);
    } while (strcmp(choice, "yes") == 0);

    printf("\nInorder Traversal : The elements in the tree are");
    inorder(root);
    printf("\nPreorder Traversal : The elements in the tree are");
    preorder(root);
    printf("\nPostorder Traversal : The elements in the tree are ");
    postorder(root);
    return 0;
}

```

SAMPLE INPUT AND OUTPUT :

Enter the element to be inserted in the tree

1

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

2

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

3

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

4

Do you want to insert another element?

no

Inorder Traversal : The elements in the tree are 1 2 3 4

Preorder Traversal : The elements in the tree are 1 2 3 4

Postorder Traversal : The elements in the tree are 4 3 2 1

4.Binary Search Tree – Search

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct tnode

{

    int data;

    struct tnode *leftc;

    struct tnode *rightc;
};

/* Insert an element into the BST */
void insert(struct tnode **s, int num)
{
    if (*s == NULL)
    {
        *s = (struct tnode*)malloc(sizeof(struct tnode));
        (*s)->data = num;
        (*s)->leftc = NULL;
        (*s)->rightc = NULL;
        return;
    }
}
```

```

    if (num <= (*s)->data)
        insert(&((*s)->leftc), num);
    else
        insert(&((*s)->rightc), num);
}

```

/* Inorder Traversal */

```

void inorder(struct tnode *s)

```

```

{
    if (s != NULL)
    {
        inorder(s->leftc);
        printf("%d ", s->data);
        inorder(s->rightc);
    }
}

```

/* Search for an element in BST */

```

int search(struct tnode *s, int ele)

```

```

{
    if (s == NULL)
        return 0;

    if (s->data == ele)
        return 1;
    else if (ele < s->data)
        return search(s->leftc, ele);
    else
        return search(s->rightc, ele);
}

```

```

int main()

```

```

{
    struct tnode *root = NULL;
    int num, ele;

```

```
char choice[10];

do
{
    printf("Enter the element to be inserted in the tree\n");
    scanf("%d", &num);
    insert(&root, num);

    printf("Do you want to insert another element?\n");
    scanf("%s", choice);

} while (strcmp(choice, "yes") == 0);

printf("\nInorder Traversal : The elements in the tree are ");
inorder(root);

printf("\n\nEnter the element to be searched\n");
scanf("%d", &ele);

if (search(root, ele))
    printf("\n%d found", ele);
else
    printf("\n%d not found", ele);

return 0;
}
```

SAMPLE INPUT AND OUTPUT:

```
Enter the element to be inserted in the tree
1
Do you want to insert another element?
yes
Enter the element to be inserted in the tree
2
```

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

3

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

4

Do you want to insert another element?

no

Inorder Traversal : The elements in the tree are 1 2 3 4

Enter the element to be searched

6

6 not found

5.Binary Search Tree – Delete

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct tnode
```

```
{
```

```
    int data;
```

```
    struct tnode *leftc;
```

```
    struct tnode *rightc;
```

```
};
```

```
/* Insert an element into BST */
```

```
void insertion(struct tnode **r, int ele)
```

```
{
```

```
if (*r == NULL)
```

```
{
```

```
    *r = (struct tnode *)malloc(sizeof(struct tnode));
```

```
    (*r)->data = ele;
```

```
    (*r)->leftc = NULL;
```

```
    (*r)->rightc = NULL;
```

```
    return;
```

```
}
```

```

    if (ele <= (*r)->data)
        insertion(&((*r)->leftc), ele);
    else
        insertion(&((*r)->rightc), ele);
}

/* Inorder Traversal */
void inorder(struct tnode *r)
{

    if (r != NULL)
    {
        inorder(r->leftc);
        printf("%d ", r->data);
        inorder(r->rightc);
    }
}

/* Find minimum value node (helper for deletion) */
struct tnode* findMin(struct tnode *r)
{
    while (r->leftc != NULL){

        r = r->leftc; return r;
    }
}

/* Delete a node from BST */

struct tnode* deletenode(struct tnode *r, int item)
{
    if (r == NULL)
        return NULL;

```

```

if (item < r->data)

    r->leftc = deletenode(r->leftc, item);
else if (item > r->data)
    r->rightc = deletenode(r->rightc, item);
else
{
    /* Node with one or no child */
    if (r->leftc == NULL)
    {
        struct tnode *temp = r->rightc;
        free(r);
        return temp;
    }

    else if (r->rightc == NULL)
    {
        struct tnode *temp = r->leftc;
        free(r);
        return temp;
    }

    /* Node with two children */

    struct tnode *temp = findMin(r->rightc);
    r->data = temp->data;
    r->rightc = deletenode(r->rightc, temp->data);

}

return r;
}

/* Search for an element */
int search(struct tnode *r, int item)
{
    if (r == NULL)

        return 0;

    if (r->data == item)
        return 1;
    if (item < r->data)

```

```

        return search(r->leftc, item);
    return search(r->rightc, item);
}

int main()
{
    struct tnode *root = NULL;
    int choice, ele;

    printf("Binary Search Tree Implementation.\n\n");
    printf("Choice 1: Insert an element.\n");
    printf("Choice 2: Delete an element.\n");
    printf("Choice 3: Display.\n\n");
    while (1)
    {
        printf("Enter your choice :\n");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter the element to be inserted :\n");
                scanf("%d", &ele);
                insertion(&root, ele);
                break;

            case 2:
                if (root == NULL)
                {
                    printf("Enter the element to be deleted :\n");
                    scanf("%d", &ele);
                    printf("Tree is empty\n");
                }
                else
                {

```



```

        printf("Enter the element to be deleted :\n");
        scanf("%d", &ele);
        if (search(root, ele))

            root = deletenode(root, ele);
        else
            printf("Data to be deleted, not found\n");
    }

    break;

case 3:

    if (root == NULL)

        printf("There are no elements in the BST.\n");
    else
    {

        inorder(root);
        printf("\n");
    }

    break;

default:

    return 0; /* Exit */

}

}

}

```

SAMPLE INPUT AND OUTPUT:

Choice 1: Insert an element.

Choice 2: Delete an element.

Choice 3: Display.

Enter your choice :

3

There are no elements in the BST.

Enter your choice :

2

Enter the element to be deleted :

12

Tree is empty

Enter your choice :

1

Enter the element to be inserted :

4

Enter your choice :

1

Enter the element to be inserted :

10

Enter your choice :

1

Enter the element to be inserted :

3

Enter your choice :

3

3 4 10

Enter your choice :

2

Enter the element to be deleted :

3

Enter your choice :

3

4 10

Enter your choice :

1

Enter the element to be inserted :

12

Enter your choice :

1

Enter the element to be inserted :

45

Enter your choice :

3

4 10 12 45

Enter your choice :

4

6.Binary Search Tree - Leaf Nodes

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct tnode
```

```
{
```

```
    int data;
```

```
    struct tnode *leftc;
```

```
    struct tnode *rightc;
```

```
};
```

```
/* Function to insert an element into BST */
```

```
void insert(struct tnode **s, int num)
```

```
{
```

```
    if (*s == NULL)
```

```
    {
```

```
    *s = (struct tnode*)malloc(sizeof(struct tnode));
    (*s)->data = num;
    (*s)->leftc = NULL;
    (*s)->rightc = NULL;
    return;
}
```

```
if (num <= (*s)->data)
    insert(&((*s)->leftc), num);
else
    insert(&((*s)->rightc), num);
}
```

```
/* Function to find the minimum value in BST */
int findMin(struct tnode *s)
{
    if (s == NULL)
        return -1;

    else if (s->leftc == NULL)
        return s->data;
    else
        return findMin(s->leftc);
}
```

```
/* Function to find the maximum value in BST */
int findMax(struct tnode *s)
{
    if (s == NULL)
        return -1;

    else if (s->rightc == NULL)
        return s->data;
    else
        return findMax(s->rightc);
}
```

```
/* Function to count total number of nodes */
int countNodes(struct tnode *s)
{
    if (s == NULL)
        return 0;
    else
        return (1 + countNodes(s->leftc) + countNodes(s->rightc));
}
```

```
/* Function to count total number of leaf nodes */

int countLeaves(struct tnode *s)
{
    if (s == NULL)
        return 0;

    else if (s->leftc == NULL && s->rightc == NULL)
        return 1;
    else
        return (countLeaves(s->leftc) + countLeaves(s->rightc));
}
```

```
int main()
{
    struct tnode *root = NULL;
    int num;
    char choice[10];

    do
    {
        printf("Enter the element to be inserted in the tree\n");
        scanf("%d", &num);
        insert(&root, num);

        printf("Do you want to insert another element? (yes/no)\n");
        scanf("%s", choice);
    }
```

```
    } while (strcmp(choice, "yes") == 0);

    printf("\nMinimum value in the BST: %d", findMin(root));
    printf("\nMaximum value in the BST: %d", findMax(root));
    printf("\nTotal number of nodes: %d", countNodes(root));
    printf("\nTotal number of leaf nodes: %d\n", countLeaves(root));

    return 0;
}
```

SAMPLE INPUT AND OUTPUT:

Enter the element to be inserted in the tree

5

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

7

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

3

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

1

Do you want to insert another element?

no

The elements in the tree are 1 3 5 7

The number of leaf nodes in the tree is 2

7.Count leaf nodes in a binary tree

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct tnode
{
    int data;

    struct tnode *leftc;
    struct tnode *rightc;
};

/* Function to insert an element into BST */
void insert(struct tnode **s, int num)
{
    if (*s == NULL)
    {
        *s = (struct tnode*)malloc(sizeof(struct tnode));
        (*s)->data = num;
        (*s)->leftc = NULL;
        (*s)->rightc = NULL;
        return;
    }

    if (num <= (*s)->data)
        insert(&((*s)->leftc), num);
    else
        insert(&((*s)->rightc), num);
}

/* Function to calculate the height of the tree */
int height(struct tnode *s)
{
    if (s == NULL)
```

```

        return 0;
    else
    {
        int lheight = height(s->leftc);
        int rheight = height(s->rightc);

        if (lheight > rheight)
            return (lheight + 1);
        else
            return (rheight + 1);
    }
}

/* Function to create a mirror image of the tree */
void mirror(struct tnode *s)
{
    if (s == NULL)
        return;
    else
    {
        struct tnode *temp;

        /* Recur for subtrees */
        mirror(s->leftc);
        mirror(s->rightc);

        /* Swap the pointers */
        temp = s->leftc;
        s->leftc = s->rightc;
        s->rightc = temp;
    }
}

/* Inorder traversal to display the tree */
void inorder(struct tnode *s)
{

```



```

    if (s != NULL)
    {
        inorder(s->leftc);
        printf("%d ", s->data);
        inorder(s->rightc);
    }
}

int main()

{
    struct tnode *root = NULL;
    int num;
    char choice[10];

    do
    {
        printf("Enter the element to be inserted in the tree\n");
        scanf("%d", &num);

        insert(&root, num);
        printf("Do you want to insert another element? (yes/no)\n");
        scanf("%s", choice);

    } while (strcmp(choice, "yes") == 0);
    printf("\nInorder Traversal of Original Tree: ");
    inorder(root);

    printf("\nHeight of the tree: %d", height(root));
    mirror(root);

    printf("\nInorder Traversal after Mirroring: ");
    inorder(root);
    printf("\n");
    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the element to be inserted in the tree

45

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

61

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

79

Do you want to insert another element?

yes

Enter the element to be inserted in the tree

27

Do you want to insert another element?

no

Leaf count of the tree is 2

MODULE 6 – Graphs And Traversals

1. Write a program to obtain the adjacency matrix representation of a graph .

PROGRAM:

```
#include <stdio.h>

#include <string.h>

int main() {

    int n, e;

    int adj[100][100] = {0};

    char type[5];

    printf("Please enter the number of nodes in the graph\n");

    scanf("%d", &n);

    printf("Please enter the number of edges in the graph\n");

    scanf("%d", &e);

    printf("Is the graph directed\n");

    scanf("%s", type);

    for (int i = 0; i < e; i++) {

        int u, v, w;

        printf("Enter the start node, end node and weight of edge no %d\n", i);

        scanf("%d %d %d", &u, &v, &w);

        adj[u][v] = w;

        // If graph is undirected, add reverse edge also

        if (strcmp(type, "no") == 0) {

            adj[v][u] = w;

        }

    }

}
```

```

printf("Adjacency Matrix Representation:\n");

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d", adj[i][j]);

        if (j != n - 1)
            printf(" ");
    }
    printf("\n");
}

return 0;
}

```

SAMPLE INPUT AND OUTPUT 1:

Please enter the number of nodes in the graph

4

Please enter the number of edges in the graph

4

Is the graph directed

no

Enter the start node, end node and weight of edge no 0

0 1 7

Enter the start node, end node and weight of edge no 1

2 3 5

Enter the start node, end node and weight of edge no 2

1 2 3

Enter the start node, end node and weight of edge no 3

3 0 6

Adjacency Matrix Representation:

0 7 0 6

7 0 3 0

0 3 0 5

6 0 5 0

SAMPLE INPUT AND OUTPUT 2:

Please enter the number of nodes in the graph

4

Please enter the number of edges in the graph

3

Is the graph directed

yes

Enter the start node, end node and weight of edge no 0

0 1 3

Enter the start node, end node and weight of edge no 1

0 2 4

Enter the start node, end node and weight of edge no 2

1 2 5

Adjacency Matrix Representation:

0 3 4 0

0 0 5 0

0 0 0 0

0 0 0 0

2. Write a program to obtain the adjacency list representation of a graph from its adjacency matrix representation.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

int main() {

    int n, e;

    int adj[100][100] = {0};

    char type[5];

    printf("Please enter the number of nodes in the graph\n");

    scanf("%d", &n);

    printf("Please enter the number of edges in the graph\n");

    scanf("%d", &e);

    printf("Is the graph directed\n");

    scanf("%s", type);

    // Input edges and build adjacency matrix

    for (int i = 0; i < e; i++) {

        int u, v, w;

        printf("Enter the start node, end node and weight of edge no %d\n", i);

        scanf("%d %d %d", &u, &v, &w);

        adj[u][v] = w;

        // If undirected, add reverse edge

        if (strcmp(type, "no") == 0) {

            adj[v][u] = w;

        }

    }

}
```

```

    }

    // Print adjacency matrix

    printf("Adjacency Matrix Representation:\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d", adj[i][j]);

            if (j != n - 1)
                printf(" ");
        }

        printf("\n");
    }

    // Convert to adjacency list and print

    printf("Adjacency List Representation:\n");

    for (int i = 0; i < n; i++) {
        int hasEdge = 0;

        // Check if node has any outgoing edges

        for (int j = 0; j < n; j++) {
            if (adj[i][j] != 0) {
                hasEdge = 1;

                break;
            }
        }

        if (hasEdge) {
            printf("Node %d is connected to the following nodes:\n", i);

            for (int j = 0; j < n; j++) {

```

```

        if (adj[i][j] != 0) {
            printf("Node %d with edge weight %d\n", j, adj[i][j]);
        }
    }
}
}
return 0;
}

```

SAMPLE INPUT AND OUTPUT 1:

Please enter the number of nodes in the graph

4

Please enter the number of edges in the graph

4

Is the graph directed

no

Enter the start node, end node and weight of edge no 0

0 1 7

Enter the start node, end node and weight of edge no 1

2 3 5

Enter the start node, end node and weight of edge no 2

1 2 4

Enter the start node, end node and weight of edge no 3

3 0 6

Adjacency Matrix Representation:

0 7 0 6

7 0 4 0

0 4 0 5

6 0 5 0

Adjacency List Representation:

Node 0 is connected to the following nodes:

Node 1 with edge weight 7

Node 3 with edge weight 6

Node 1 is connected to the following nodes:

Node 0 with edge weight 7

Node 2 with edge weight 4

Node 2 is connected to the following nodes:

Node 1 with edge weight 4

Node 3 with edge weight 5

Node 3 is connected to the following nodes:

Node 0 with edge weight 6

Node 2 with edge weight 5

SAMPLE INPUT AND OUTPUT 2:

Please enter the number of nodes in the graph

4

Please enter the number of edges in the graph

3

Is the graph directed

yes

Enter the start node, end node and weight of edge no 0

0 1 3

Enter the start node, end node and weight of edge no 1

0 2 4

Enter the start node, end node and weight of edge no 2

1 2 5

Adjacency Matrix Representation:

0 3 4 0

0 0 5 0

0 0 0 0

0 0 0 0

Adjacency List Representation:

Node 0 is connected to the following nodes:

Node 1 with edge weight 3

Node 2 with edge weight 4

Node 1 is connected to the following nodes:

Node 2 with edge weight 5

3. Write a program to implement Depth First Traversal for a given graph.

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int adj[100][100];
```

```
int visited[100];
```

```
int n;
```

```

// DFS function

void dfs(int v) {
    visited[v] = 1;
    printf("%d ", v);
    for (int i = 0; i < n; i++) {
        if (adj[v][i] != 0 && !visited[i]) {
            dfs(i);
        }
    }
}

int main() {
    int e;

    char type[5];

    printf("Enter the number of nodes in the graph\n");
    scanf("%d", &n);

    printf("Enter the number of edges in the graph\n");
    scanf("%d", &e);

    printf("Is the graph directed(yes/no)\n");
    scanf("%s", type);

    // Initialize matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    // Input edges
    for (int i = 0; i < e; i++) {

```

```

    int u, v, w;

    printf("Enter the start node, end node and weight of edge no %d\n", i);

    scanf("%d %d %d", &u, &v, &w);

    adj[u][v] = w;

    if (strcmp(type, "no") == 0) {

        adj[v][u] = w;

    }

}

// Print adjacency matrix

printf("Adjacency Matrix Representation:\n");

for (int i = 0; i < n; i++) {

    for (int j = 0; j < n; j++) {

        printf("%d", adj[i][j]);

        if (j != n - 1)

            printf(" ");

    }

    printf("\n");

}

// Print adjacency list

printf("Adjacency List Representation:\n");

for (int i = 0; i < n; i++) {

    int hasEdge = 0;

    for (int j = 0; j < n; j++) {

        if (adj[i][j] != 0) {

            hasEdge = 1;

```

```

        break;
    }
}

if (hasEdge) {
    printf("Node %d is connected to the following nodes:\n", i);
    for (int j = 0; j < n; j++) {
        if (adj[i][j] != 0) {
            printf("Node %d with edge weight %d\n", j, adj[i][j]);
        }
    }
}
}

int start;

printf("Enter the starting node / vertex for depth first traversal\n");
scanf("%d", &start);

// Initialize visited array
for (int i = 0; i < n; i++)
    visited[i] = 0;

printf("Depth First Traversal starting from node %d\n", start);

// Start DFS from given node
dfs(start);

// Handle disconnected components (DFS Forest)
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i);
    }
}

```

```
    }  
}  
return 0;  
}
```

SAMPLE INPUT AND OUTPUT 1:

Enter the number of nodes in the graph

4

Enter the number of edges in the graph

5

Is the graph directed(yes/no)

no

Enter the start node, end node and weight of edge no 0

0 1 4

Enter the start node, end node and weight of edge no 1

0 3 6

Enter the start node, end node and weight of edge no 2

1 2 2

Enter the start node, end node and weight of edge no 3

1 3 1

Enter the start node, end node and weight of edge no 4

2 3 5

Adjacency Matrix Representation:

0 4 0 6

4 0 2 1

0 2 0 5

6 1 5 0

Adjacency List Representation:

Node 0 is connected to the following nodes:

Node 1 with edge weight 4

Node 3 with edge weight 6

Node 1 is connected to the following nodes:

Node 0 with edge weight 4

Node 2 with edge weight 2

Node 3 with edge weight 1

Node 2 is connected to the following nodes:

Node 1 with edge weight 2

Node 3 with edge weight 5
Node 3 is connected to the following nodes:
Node 0 with edge weight 6
Node 1 with edge weight 1
Node 2 with edge weight 5
Enter the starting node / vertex for depth first traversal
0
Depth First Traversal starting from node 0
0 1 2 3

SAMPLE INPUT AND OUTPUT 2:

Enter the number of nodes in the graph
4
Enter the number of edges in the graph
5
Is the graph directed(yes/no)
yes
Enter the start node, end node and weight of edge no 0
0 1 4
Enter the start node, end node and weight of edge no 1
0 3 6
Enter the start node, end node and weight of edge no 2
2 1 2
Enter the start node, end node and weight of edge no 3
1 3 1
Enter the start node, end node and weight of edge no 4
3 2 5
Adjacency Matrix Representation:
0 4 0 6
0 0 0 1
0 2 0 0
0 0 5 0
Adjacency List Representation:
Node 0 is connected to the following nodes:
Node 1 with edge weight 4
Node 3 with edge weight 6
Node 1 is connected to the following nodes:
Node 3 with edge weight 1
Node 2 is connected to the following nodes:
Node 1 with edge weight 2

Node 3 is connected to the following nodes:
Node 2 with edge weight 5
Enter the starting node / vertex for depth first traversal
0
Depth First Traversal starting from node 0
0 1 3 2

4. Write a program to implement Breadth First Traversal for a given graph.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

int adj[100][100];

int visited[100];

int queue[100];

int n;

// BFS function

void bfs(int start) {

    int front = 0, rear = 0;

    queue[rear++] = start;

    visited[start] = 1;

    while (front < rear) {

        int v = queue[front++];

        printf("%d ", v);

        for (int i = 0; i < n; i++) {

            if (adj[v][i] == 1 && !visited[i]) {

                queue[rear++] = i;

                visited[i] = 1;

            }

        }

    }

}
```



```

        }

    }

}

int main() {

    int e;

    char type[5];

    printf("Enter the number of nodes in the graph\n");

    scanf("%d", &n);

    printf("Enter the number of edges in the graph\n");

    scanf("%d", &e);

    printf("Is the graph directed(yes/no)\n");

    scanf("%s", type);

    // Initialize adjacency matrix

    for (int i = 0; i < n; i++)

        for (int j = 0; j < n; j++)

            adj[i][j] = 0;

    // Input edges

    for (int i = 0; i < e; i++) {

        int u, v;

        printf("Enter the start node and end node of edge no %d\n", i);

        scanf("%d %d", &u, &v);

        adj[u][v] = 1;

        if (strcmp(type, "no") == 0) {

            adj[v][u] = 1;

```

```

        }
    }

    int start;

    printf("Enter the starting node / vertex for breadth first traversal\n");

    scanf("%d", &start);

    // Initialize visited array

    for (int i = 0; i < n; i++)

        visited[i] = 0;

    printf("Breadth First Traversal starting from node %d\n", start);

    // BFS from starting node

    bfs(start);

    // Handle disconnected components (BFS Forest)

    for (int i = 0; i < n; i++) {

        if (!visited[i]) {

            bfs(i);

        }

    }

    return 0;

}

```

SAMPLE INPUT AND OUTPUT:

```

Enter the number of nodes in the graph
4
Enter the number of edges in the graph
5
Is the graph directed(yes/no)
no

```

Enter the start node and end node of edge no 0
0 1
Enter the start node and end node of edge no 1
0 3
Enter the start node and end node of edge no 2
1 2
Enter the start node and end node of edge no 3
1 3
Enter the start node and end node of edge no 4
2 3
Enter the starting node / vertex for breadth first traversal
0
Breadth First Traversal starting from node 0
0 1 3 2

SAMPLE INPUT AND OUTPUT 2:

Enter the number of nodes in the graph
4
Enter the number of edges in the graph
5
Is the graph directed(yes/no)
yes
Enter the start node and end node of edge no 0
0 1
Enter the start node and end node of edge no 1
3 0
Enter the start node and end node of edge no 2
1 2
Enter the start node and end node of edge no 3
1 3
Enter the start node and end node of edge no 4
3 2
Enter the starting node / vertex for breadth first traversal
0
Breadth First Traversal starting from node 0
0 1 2 3

5. Wedding of Sultan

PROGRAM:

```
#include <stdio.h>

#include <string.h>

int main() {

    char sequence[1000];

    int degree[26] = {0};    // To store trail counts for A-Z

    int exists[26] = {0};    // To track which sprinklers are in the garden

    char stack[1000];

    int top = -1;

    if (scanf("%s", sequence) == 1) {

        int len = strlen(sequence);

        for (int i = 0; i < len; i++) {

            int current = sequence[i] - 'A';

            exists[current] = 1;

            // If stack is not empty and we are returning to the parent

            if (top >= 0 && stack[top] == sequence[i]) {

                top--; // Move back up the tree

            }

            else {

                // Moving down to a new sprinkler

                if (top >= 0) {

                    // There is a trail between stack[top] and sequence[i]

                    degree[stack[top] - 'A']++;

                    degree[current]++;

                }

            }

        }

    }

}
```

```

    }

    // Push current sprinkler onto stack
    stack[++top] = sequence[i];
}

}

// Print results in lexicographical order
for (int i = 0; i < 26; i++) {
    if (exists[i]) {
        printf("%c = %d\n", i + 'A', degree[i]);
    }
}

}

return 0;
}

```

SAMPLE INPUT:

AEFFGGEBDDCCBA

SAMPLE OUTPUT:

A = 2

B = 3

C = 1

D = 1

E = 3

F = 1

G = 1

MODULE 7 – Searching And Simple Sorting Techniques

1. Linear Search

PROGRAM:

```
#include <stdio.h>
int main() {
    int n, i, l, found = 0;

    printf("Enter the total number of tickets:\n");

    scanf("%d", &n);

    int ticket;

    // Input ticket numbers

    for(i = 0; i < n; i++) {
        printf("Enter the ticket number %d:\n", i + 1);
        scanf("%d", &tickets[i]);
    }

    // Display ticket numbers

    printf("The ticket numbers are:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", tickets[i]);
    }

    // Input search number
    printf("\nEnter the ticket number to be searched:\n");
    scanf("%d", &l);

    // Linear Search

    for(i = 0; i < n; i++) {
        if(tickets[i] == l) {
            found = 1;
            printf("The ticket number %d is found at position %d\n", l, i + 1);
            printf("Congratulations! You have won the lottery\n");
            break;
        }
    }

    // If not found
    if(found == 0) {
        printf("Sorry the ticket number %d is not there. Better luck next time!\n", l);
    }
}
```

```
        return 0;  
    }
```

SAMPLE INPUT AND OUTPUT 1:

Enter the total number of tickets:

5

Enter the ticket number 1:

4521

Enter the ticket number 2:

3589

Enter the ticket number 3:

147852

Enter the ticket number 4:

2365

Enter the ticket number 5:

8965

The ticket numbers are:

4521 3589 147852 2365 8965

Enter the ticket number to be searched:

8965

The ticket number 8965 is found at position 5

Congratulations! You have won the lottery

SAMPLE INPUT AND OUTPUT 2:

Enter the total number of tickets:

5

Enter the ticket number 1:

4521

Enter the ticket number 2:

3589

Enter the ticket number 3:

147852

Enter the ticket number 4:

2365

Enter the ticket number 5:

8965

The ticket numbers are:

4521 3589 147852 2365 8965

Enter the ticket number to be searched:

1254

Sorry the ticket number 1254 is not there. Better luck next time!

2. Binary Search

PROGRAM:

```
#include <stdio.h>

int main() {
    int n, i, key;
    printf("Enter the total number of jails in Humayun's Place:\n");
    scanf("%d", &n);

    int jails[n];

    // Input sorted jail numbers

    for(i = 0; i < n; i++) {
        printf("Enter the jail number %d\n", i + 1);
        scanf("%d", &jails[i]);
    }

    printf("Enter the clue given by Humayun:\n");
    scanf("%d", &key);

    // Binary Search
    int low = 0, high = n - 1, mid;
    int found = 0;

    while(low <= high) {
        mid = (low + high) / 2;

        if(jails[mid] == key) {
            found = 1;
            break;
        } else if(key > jails[mid]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // Output result
    if(found == 1) {
        printf("Hurray! The King rescued the Queen\n");
    } else {
        printf("The King has been fooled by Humayun\n");
    }
}
```



```
        return 0;
    }
```

SAMPLE INPUT AND OUTPUT :

Enter the total number of jails in Humayun's Place:

5

Enter the jail number 1

6

Enter the jail number 2

89

Enter the jail number 3

91

Enter the jail number 4

105

Enter the jail number 5

200

Enter the clue given by Humayun:

105

Hurray!The King rescued the Queen.

3.Selection Sort

PROGRAM:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j, minIndex, temp;
```

```
    printf("Enter the total number of students in class:\n");
```

```
    scanf("%d", &n);
```

```
    int height[n];
```

```
// Input heights
for(i = 0; i < n; i++) {
    printf("Enter the height of student %d:\n", i + 1);
    scanf("%d", &height[i]);
}

// Before sorting
printf("Student height order before sorting:\n");
for(i = 0; i < n; i++) {
    printf("%d ", height[i]);
}
printf("\n");

// Selection Sort
for(i = 0; i < n - 1; i++) {
    minIndex = i;
    for(j = i + 1; j < n; j++) {
        if(height[j] < height[minIndex]) {
            minIndex = j;
        }
    }
}

// Swap
temp = height[i];
height[i] = height[minIndex];
height[minIndex] = temp;

// Print after each iteration
printf("Height order of students after iteration %d:\n", i + 1);
for(j = 0; j < n; j++) {
    printf("%d ", height[j]);
}
printf("\n");
```

```
    }  
    // Final output  
    printf("After final comparison of all students, the height order becomes:\n");  
    for(i = 0; i < n; i++) {  
        printf("%d ", height[i]);  
    }  
    return 0;  
}
```

SAMPLE INPUT AND OUTPUT:

Enter the total number of students in class

6

Enter the height of student 1

480

Enter the height of student 2

160

Enter the height of student 3

250

Enter the height of student 4

130

Enter the height of student 5

80

Enter the height of student 6

50

Student's height order before sorting:

480 160 250 130 80 50

Height order of students after iteration 1

50 160 250 130 80 480

Height order of students after iteration 2

50 80 250 130 160 480

Height order of students after iteration 3

50 80 130 250 160 480

Height order of students after iteration 4

50 80 130 160 250 480

Height order of students after iteration 5

50 80 130 160 250 480

After final comparison of all students, the height order becomes:

50 0 130 160 250 480

4. Write a program to perform insertion sort on an array of n elements.

PROGRAM:

```
#include <stdio.h>
int main() {
    int n, i, j, key;
    printf("Enter the total number of cards\n");
    scanf("%d", &n);
    int cards[n];
    // Input cards
    for(i = 0; i < n; i++) {
        printf("Enter card number %d\n", i + 1);
        scanf("%d", &cards[i]);
    }
    // Before sorting
    printf("Cards before being sorted:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", cards[i]);
    }
    printf("\n");
    // Insertion Sort
    for(i = 1; i < n; i++) {
        key = cards[i];
        j = i - 1;
        while(j >= 0 && cards[j] > key) {
            cards[j + 1] = cards[j];
            j--;
        }
    }
}
```

```
cards[j + 1] = key;
// Print after each iteration
printf("Arrangement of cards after iteration %d\n", i);
for(int k = 0; k < n; k++) {
    printf("%d ", cards[k]);
}
printf("\n");
}
// Final sorted order
printf("Finally the sorted card order is:\n");
for(i = 0; i < n; i++) {
    printf("%d ", cards[i]);
}
return 0;
}
```

SAMPLE INPUT AND OUTPUT:

Enter the total number of cards

13

Enter card number 1

7

Enter card number 2

11

Enter card number 3

9

Enter card number 4

12

Enter card number 5

1

Enter card number 6

8

Enter card number 7

4

Enter card number 8

5

Enter card number 9

2

Enter card number 10

3

Enter card number 11

13

Enter card number 12

10

Enter card number 13

6

Cards before being sorted:

7 11 9 12 1 8 4 5 2 3 13 10 6

Arrangement of cards after iteration 1

7 11 9 12 1 8 4 5 2 3 13 10 6

Arrangement of cards after iteration 2

7 9 11 12 1 8 4 5 2 3 13 10 6

Arrangement of cards after iteration 3

7 9 11 12 1 8 4 5 2 3 13 10 6

Arrangement of cards after iteration 4

1 7 9 11 12 8 4 5 2 3 13 10 6

Arrangement of cards after iteration 5

1 7 8 9 11 12 4 5 2 3 13 10 6

Arrangement of cards after iteration 6

1 4 7 8 9 11 12 5 2 3 13 10 6

Arrangement of cards after iteration 7

1 4 5 7 8 9 11 12 2 3 13 10 6

Arrangement of cards after iteration 8

1 2 4 5 7 8 9 11 12 3 13 10 6

Arrangement of cards after iteration 9

1 2 3 4 5 7 8 9 11 12 13 10 6

Arrangement of cards after iteration 10

1 2 3 4 5 7 8 9 11 12 13 10 6

Arrangement of cards after iteration 11

1 2 3 4 5 7 8 9 10 11 12 13 6

Arrangement of cards after iteration 12

1 2 3 4 5 6 7 8 9 10 11 12 13

Finally the sorted card order is:

1 2 3 4 5 6 7 8 9 10 11 12 13

5.Bubble sort :

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
// Create node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
// Print list
void printList(struct Node* head) {
    struct Node* temp = head;
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
// Bubble Sort
void bubbleSort(struct Node* head, int n) {
```

```

int i, swapped;
struct Node *ptr, *last = NULL;
for(i = 0; i < n - 1; i++) {
    swapped = 0;
    ptr = head;
    while(ptr->next != last) {
        if(ptr->data > ptr->next->data) {
            int temp = ptr->data;
            ptr->data = ptr->next->data;
            ptr->next->data = temp;
            swapped = 1;
        }
        ptr = ptr->next;
    }
    // Move last pointer back
    last = ptr;
    // Print only if at least one swap happened
    if(swapped == 1) {
        printf("Height order of students after iteration %d\n", i + 1);
        printList(head);
    } else {
        break; // Stop if already sorted
    }
}
}

int main() {
    int n, i, value;
    printf("How many students can be seated in a bench?\n");
    scanf("%d", &n);
    struct Node *head = NULL, *temp = NULL, *newNode;
    for(i = 0; i < n; i++) {
        printf("Enter the height of student %d\n", i + 1);
        scanf("%d", &value);
        newNode = createNode(value);
        if(head == NULL) {
            head = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    printf("Height order of students before sorting:\n");

```



```
    printList(head);
    bubbleSort(head, n);
    printf("Final sorting of students in a bench are:\n");
    printList(head);
    return 0;
}
```

SAMPLE INPUT AND OUTPUT:

How many students can be seated in a bench?

5

Enter the height of student 1

130

Enter the height of student 2

180

Enter the height of student 3

120

Enter the height of student 4

80

Enter the height of student 5

60

Height order of students before sorting:

130 180 120 80 60

Height order of students after iteration 1

130 120 80 60 180

Height order of students after iteration 2

120 80 60 130 180

Height order of students after iteration 3

80 60 120 130 180

Height order of students after iteration 4

60 80 120 130 180

Final sorting of students in a bench are:

60 80 120 130 180

6.Rotated Sorted Array

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Create node
```

```
struct Node* createNode(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
// Get middle node
```

```
struct Node* getMid(struct Node* start, struct Node* end) {  
    struct Node *slow = start, *fast = start;  
    while(fast != end && fast->next != end) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow;  
}
```

```
// Search in rotated sorted linked list
```

```
int search(struct Node* head, int key) {  
    struct Node *start = head, *end = NULL;  
    while(start != end) {  
        struct Node* mid = getMid(start, end);  
        // Find index of mid  
        struct Node* temp = head;  
        int midIndex = 0;  
        while(temp != mid) {  
            temp = temp->next;  
            midIndex++;  
        }  
        if(mid->data == key)  
            return midIndex;  
        // Check if left half is sorted  
        if(start->data <= mid->data) {  
            if(key >= start->data && key < mid->data)  
                end = mid;  
            else {  
                start = mid->next;  
            }  
        }  
        // Right half sorted  
        else {  
            struct Node* last = start;
```

```

        while(last->next != end)
            last = last->next;
        if(key > mid->data && key <= last->data)
            start = mid->next;
        else
            end = mid;
    }
}

return -1;
}

int main() {
    int n, i, value, key;
    printf("Enter the number of elements in the array\n");

    scanf("%d", &n);
    struct Node *head = NULL, *temp = NULL, *newNode;
    printf("Enter the elements in the array\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &value);
        newNode = createNode(value);
        if(head == NULL) {
            head = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    printf("Enter the element to be searched\n");
    scanf("%d", &key);
    int result = search(head, key);
    if(result != -1)
        printf("The index of element %d in the array is %d", key, result);
    else
        printf("The element %d is not found in the array", key);

    return 0;
}

```

SAMPLE INPUT AND OUTPUT:

Enter the number of elements in the array

12

Enter the elements in the array

15

16

19

20

25

1

3

4

5

7

10

14

Enter the element to be searched

5

The index of element 5 in the array is 8